

SOVEREIGN STACK

A ground-up IT education for the age of AI

VOLUME 1

Home Node

Build Your Own Local AI at Home,
Reachable Securely From Anywhere, Depending on Nobody

A foundation starter kit for one person and one machine: from bare hardware to an AI agent that runs on the box you own, reached over the internet, with no cloud doing its thinking or holding its data, and the networking and security to understand every single step.

A multi-volume series. Volume 1 covers the home, one person and one node.
Later volumes carry the same method outward: the school, the university, the city.

First Edition

A living field manual, a map, not scripture.

Contents

Cover	1
A note before you start	4
How to read this book	5
THE MAP: WHAT YOU WILL LEARN, AND WHERE YOU ARE	6
The thesis: own your ground	8
A day with a home node	9
THE COMPANION APP, AND WHY THIS IS ITS MANUAL	11
What it looks like, and what it does	11
PART 0: THE MAP	15
0.1 The abstraction ladder	15
0.2 LAN versus WAN: the most important distinction in this book	16
0.3 Two security models: the vault and the drawer	17
0.4 What “running a server” actually means	17
0.5 Files, processes, and ports: the three things you will always check	17
0.6 A worked debugging story, start to finish	18
PART 1: THE HARDWARE, AND THE MEMORY-BANDWIDTH LENS	19
1.1 The only question that matters: what will it run?	19
1.2 Memory bandwidth is the lens	19
1.3 The bandwidth timeline: how technology lined up	20
1.4 RAM versus VRAM versus unified memory	21
1.5 Choosing a chassis: mini-PC versus desktop versus laptop	21
1.6 A worked purchase: from “I want this” to a parts list	22
1.7 Storage, and the one boring essential	23
1.8 Power, heat, and what it costs to run all year	23
1.9 Buying used safely	23
PART 2: THE OPERATING SYSTEM, LINUX AS HOME BASE	25
2.1 Why Linux, and the one law of this book	25
2.2 First contact: the shell	26
2.3 Users, root, and sudo	26
2.4 Updates: the cheapest security you can buy	27
2.5 SSH: reaching the node safely	27
2.6 systemd: making things run and survive	28
2.7 Packages and the filesystem, a little deeper	29
2.8 When an update breaks something: the recovery mindset	29
PART 3: NETWORKING FROM ZERO	31
3.1 The request and response dance	31
3.2 localhost versus 0.0.0.0: the bind that bites everyone	31
3.3 NAT: how one public IP serves a whole house	32
3.4 IPv4 versus IPv6, briefly and honestly	32
3.5 Finding your way around your own network	32
3.6 Wires and radio: why CAT6 and WiFi are enough for a home	33

3.7 Understanding your router, the one box that matters	34
3.8 DNS, a little deeper, and a quiet privacy leak	34
PART 4: THE LOCAL AI	35
4.1 The runner	35
4.2 Picking a model to fit your memory budget	35
4.3 Senses: local speech	36
4.4 Memory: embeddings and a vector store	36
4.5 What an “agent” actually is	36
4.6 Talking to a model well: context, system prompt, temperature	37
4.7 Choosing between models, and keeping them current	38
4.8 The honest gap: local versus cloud in 2026	38
PART 5: RETRIEVAL, AND YOUR OWN OFFLINE WIKIPEDIA	39
5.1 What retrieval (RAG) is, in one picture	39
5.2 The flagship: all of Wikipedia, offline, on your shelf	39
5.3 How it actually works	39
5.4 The budget, in 2026 euros	40
5.5 Beyond Wikipedia	41
5.6 Chunking and embedding choices, and why retrieval sometimes misses	41
5.7 Keeping your offline Wikipedia fresh	42
PART 6: MAKING IT REACHABLE, SAFELY	43
6.1 The reverse proxy: a doorman that stores nothing	43
6.2 Walkthrough A: your first page, “Hello, World,” served from your own node	43
6.3 A name and a moving target: domain plus dynamic DNS	44
6.4 The hole in the valve: port forwarding	44
6.5 HTTPS, and the key in the URL	45
6.6 Walkthrough B: page two is an entire app, the voice recorder	46
6.7 Walkthrough C: a third page that serves data, a tiny API	47
6.8 Troubleshooting reachability: works on the LAN, not on the WAN	47
6.9 The payoff: what WAN-optional actually buys you	48
PART 7: CYBERSECURITY FOUNDATIONS	50
7.1 The firewall: shut every door you do not use	50
7.2 Reading your logs, where the truth lives	50
7.3 Banning the probbers automatically	51
7.4 Threat-modelling a home node	51
7.5 Backups: the bill the cloud used to pay silently	51
7.6 The kill-switch, and the EM caveat	52
7.7 Secrets: where keys and tokens live, and how not to leak them	52
7.8 Physical security and the human layer	52
7.9 What the open internet does to an exposed port	53
7.10 Versioning: the backup that remembers every step	54
PART 8: THE SELF-SUSTAINING ECOSYSTEM	56
8.1 A society of small agents	56
8.2 How they talk	56
8.3 Logs reviewed by AI, on a schedule	56
8.4 The worked automation: a disk starts to fail	57
8.5 Keeping the ecosystem alive	57
8.6 A worked agent: your private morning digest	57

8.7 Observability: a small dashboard of your node's health	58
8.8 The reboot drill: rehearsing the power cut	59
PART 9: THE METHOD, BUILDING ALL OF THIS WITH AI	61
9.1 Bifurcation: functionality is a list of tests	61
9.2 A worked example: the voice recorder, as a feature list	61
9.3 How to talk to the model	62
9.4 When not to let the model write the code	62
9.5 A second worked example: the firewall, as behaviours	62
9.6 Where this leads: define the tests, let the machine rebuild	63
PART 10: TROUBLESHOOTING, AND THE LAYER MAP	64
10.1 The map you have been climbing all along	64
10.2 The eight layers	64
10.3 Zoom in, or zoom out: the one decision	66
10.4 Working with a model that cannot see your machine	67
10.5 A worked failure, walked the long way	68
THE BUILD PATH: THE WHOLE SEQUENCE, IN ORDER	70
HOW THIS BOOK WAS MADE	73
Why LaTeX, and why a PDF	73
A format for every job	74
Reproducing this on Arch	75
CLOSING	77
The minimal toolset	77
Make, maintain, manage	77
Is your stack sovereign? A self-test	77
What the next volumes build	78
The map is yours	78
APPENDIX: THE SPICY BIT	80

A note before you start

This book has a rule it applies to itself, so it is only fair to state it on page one: **no instruction here is guaranteed to work on your exact machine.** The number of variables (your hardware, your firmware, your distribution, your versions, your network, your country) is larger than any book can hold. What this book teaches is not a fixed list of commands to obey. It teaches you to *adapt*, by conversing your way out of trouble and reading the errors your machine hands back. That skill is the real curriculum, and Part 2 is built around it.

A second rule, which is itself one of the book's lessons, so meet it early: **nothing here has to be perfect.** This is a book of principles, not specifications. The numbers in it (bandwidths, prices, sizes, speeds) are approximations with rough tolerance windows, good enough to make a decision by and no better, because that is all a decision needs. Where a number genuinely matters, think in terms of a window ("somewhere between 95 and 112 tokens per second") rather than a false point value, and reach for a confidence interval only when the stakes justify the effort. Otherwise, approximate freely. Hardware names drift, prices move, models get replaced. When a figure changes under you, do not chase the decimal. Re-derive the *choice* from the principle, because the principle is the part that lasts. It is never about the details here. It is about which idea to apply, and when.

A style note: this book avoids the long dash. It uses commas, colons, parentheses, and the plain hyphen instead. That is a deliberate constraint, like the rest of the book.

One piece of vocabulary, used constantly from here on. When this book says **your model** or **your LLM**, it means a Large Language Model: the kind of AI (Artificial Intelligence) you can chat with. Early on it may be a model in the cloud. The whole arc of this book is to move that model onto your own machine, where it becomes a **Local Large Language Model**, an **LLLM** (three Ls, one M): yours, offline, private, answering without a vendor in the loop. That third L is the whole journey of this book, the difference between renting intelligence and owning it. And because the software stack it runs on is open source, an LLLM is not only local but largely inspectable: you can, in principle, read the code of the thing answering you, which turns "trust it" into a claim you can verify rather than a hope you extend (with one honest limit, the closed firmware and driver floor beneath all software, that the book is careful about later). The shorthand for that destination is your own *local* model, an LLLM, and reaching it is the point.

How to read this book

This is two books braided together.

The first is a **critical path**: a straight line from an empty machine to a local AI you can talk to from your phone on the far side of the planet, over HTTPS (HyperText Transfer Protocol Secure), with no rented computer in the middle doing the work and no vendor holding your data. You pay for the dumb pipe that carries the traffic, your internet connection, but for nothing that thinks or stores on your behalf, and Part 6.5 shows why even that pipe can never read what crosses it. If you only want to build the thing, follow Parts 1 through 6 and skip the rest until something breaks.

The second is a **map of the foundations**: the mental models and the security thinking that turn “I followed a tutorial” into “I understand my machine and I can fix it without asking anyone.” That is Parts 0, 7, 8, and 9. They are what make you self-reliant instead of dependent on the next tutorial.

Throughout, you will find blocks marked **Pointer**. These are not commands to run. They are things to *ask your model*, prompts you paste into whatever LLM you have to hand (a cloud one at first, your own local one later) to get unstuck, go deeper, or adapt a step to your exact machine. They are the book teaching you its real skill in miniature: when in doubt, ask well. The Pointers are the bridge between reading and doing.

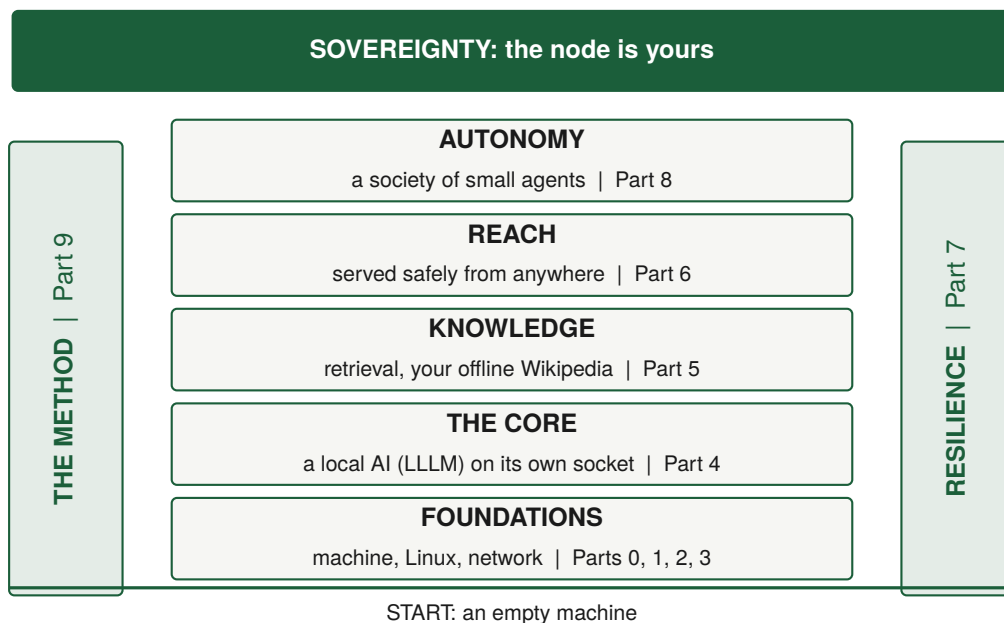
This book and a model, together. There is a third way to read this book, and it is the one it was designed for. This is a cookbook. It gives you the map, the order, the pipelines, and the reasons, the things that do not change between machines. It deliberately does not give you the exact command for your precise operating system, your hardware, your distribution version, your country’s defaults, or your language, because those change constantly, there are far too many combinations for any book to hold, and something else supplies them better: a model. The intended method is to read a step here, understand what it does and why, and then hand that step, together with your specific situation, to whatever LLM you prefer (a search engine’s built-in one, a familiar chat assistant, your own local model once you have it) and let it produce the exact commands tailored to you. The book is the recipe; the model is the kitchen that adapts the recipe to your ingredients. Neither is complete alone: the book without a model leaves you translating generic steps by hand, and a model without this book leaves you with no map of what to ask for and no judgement for when its answer is wrong. Together they are a complete, personal guide, which is why any reader, with any mainstream model and this one document, can be walked through the whole setup with very little friction.

This is also why the book teaches principles over commands so insistently. A command is a detail a model can regenerate for your exact machine in seconds. A principle (which layer is lying, local versus wide-area networking, the drawer model, the bandwidth lens, functionality as a list of tests) is the part you must carry yourself, because it is what lets you direct the model, judge its output, and recover when a tailored command still does not work. Hold the principles; let the model hold the syntax. That division is the same one Part 9 makes for building software, turned on this very book: you supply the intent and the judgement, the model supplies the labour, and the result is yours.

THE MAP: WHAT YOU WILL LEARN, AND WHERE YOU ARE

A short orientation before the journey, so that you always know where you stand on the road to owning your own ground.

Here is the first encouraging truth: the curriculum is small. It looks large because there are so many commands, but commands are details a model can regenerate for your exact machine, as the previous section argued. What you actually have to learn, and keep, is a handful of principles. Master a principle and the skill follows, because you can re-derive the skill from it whenever you need it. So the real map is not a list of a hundred tasks. It is a small set of ideas, each of which implies a whole skill.



Read the map as a building. At the base sit the **foundations**: your machine, Linux, and your network, the ground everything else rests on. On that ground you raise, in order, the **core** (a local model answering on its own socket), **knowledge** (retrieval, up to your own offline Wikipedia), **reach** (serving it safely from anywhere), and **autonomy** (a society of small agents that tend the whole thing). Holding it all up are two pillars that touch every layer: **the method** (you describe what you want and verify it with tests, and the machine does the rest) and **resilience** (you defend, back up, and version, so nothing is lost). When the building stands, the pediment is **sovereignty**: the node is yours, top to bottom.

That is also how you locate yourself. You are not “on page thirty.” You are at a layer. Can you reach your machine and read its errors by rung? You have the foundations. Is a local model answering you? You have the core. Can you ask it about your own documents with no signal? You have knowledge. Can you open it from your phone across the world, over HTTPS, privately? You have reach. Does it tend itself? You have autonomy. And the two pillars grow with you, because every layer is something you can test and recover.

The skills, and the principle behind each. Here is the same map as a checklist. Each line is a skill you will own, the principle that implies it (hold the principle and you can rebuild the skill from memory), and what it unlocks. If you can state the principle in your own words, you effectively have the skill, whether or not you recall the exact command.

- **Read your machine by layers** (files, processes, ports). Principle: every fault sits on one rung of the abstraction ladder, so find the rung. Unlocks: debugging almost anything, without a tutorial.
- **Operate Linux safely** (a normal user with sudo, regular updates, key-only SSH). Principle: least privilege, and the only door in is one you physically hold. Unlocks: a private, hardened base you reach remotely.
- **Converse your way out of trouble.** Principle: no instruction is guaranteed on your exact machine, so ask well, read the error, and climb the self-reliance ladder. Unlocks: independence from any single tutorial or vendor.
- **See your own network** (LAN versus WAN, sockets, NAT, what binds where). Principle: inside is a different country than outside, and the border is your router. Unlocks: knowing exactly what is reachable, and by whom.
- **Run a local model**, an LLM, on its own socket. Principle: size the machine to the model through the bandwidth lens. Unlocks: a private AI answering with no vendor in the loop.
- **Give it grounded knowledge** (retrieval, up to an offline Wikipedia). Principle: a stable engine plus a knowledge source you own, where you retrieve and then generate. Unlocks: an assistant that knows your world, offline.
- **Serve it safely** (one reverse-proxy front door, HTTPS, dynamic DNS, a single forwarded port). Principle: one guarded door, everything else on localhost, and HTTPS the instant you cross to the WAN. Unlocks: your node reachable from anywhere, privately.
- **Defend and recover** (firewall, logs, bans, secrets, backups, versioning). Principle: expose the minimum, layer your defenses, and make every loss recoverable. Unlocks: a node that shrugs off both probing and failure.
- **Compose small agents.** Principle: each agent does one thing reliably, and the intelligence emerges from how they connect. Unlocks: a node that tends itself.
- **Describe and verify.** Principle: own the definition of “correct” (the tests), not the code, and let the machine write the rest. Unlocks: building, and rebuilding, anything, at any scale.

Ten skills, far fewer principles once you notice how they rhyme, and a single destination. For the exact order in which to build them, hands on, see “The Build Path” near the end of the book. For the test of whether you have arrived, see the self-test in the closing. This map is the territory between those two: keep it in mind, and you will always know both where you are standing and what the next layer up is.

The thesis: own your ground

The cloud sold you a quiet trade: convenience now, dependence forever. Your photos, your notes, your conversations, your thinking, all of it living on someone else's computer, under someone else's terms of service, one policy change or outage or price rise away from being gone or turned against you.

It did not have to be that way, and in 2026 it no longer has to be. We are at a genuinely new point: the hardware to run real AI now fits on a shelf, sips power, and costs less than a decent laptop. A used graphics card from 2020 with 24 GB of memory, the going rate for which in the EU (European Union) sat between roughly 750 and 950 euros through 2026, runs capable models entirely on your own desk. The software to run it is free and open. What was missing was the map from "I am interested" to "it is running in my house and I control all of it." This book is that map.

This book calls the result a **home node**: a computer you own, on a network you own, running services you own, reachable from anywhere yet dependent on nobody for the thinking it does or the data it holds. The internet is the road you take to reach it from outside, not something it runs on: pull the cable to the outside world and your node keeps running as if nothing happened, because everything it needs was always local.

This book assumes one thing you very likely already have: access to an AI you can ask questions, a cloud chatbot to begin with. It assumes no technical background beyond the ability to read this and to ask that AI for help; everything else it teaches. The whole arc is to take the intelligence you are currently renting and anchor it to ground you own, so that the single most useful tool of this era becomes something no policy change, price rise, or outage can take from you or quietly alter underneath you. The book is the map and the reasons, the parts that do not change between machines; the AI in your hand is the kitchen that turns each step into the exact commands for your exact setup. Together they are enough, with no prior expertise required, and that is the point: this is meant to be a first, complete, self-contained starting map for anyone, a curious teenager included, who wants to understand and run their own information technology rather than wait to be taught it.

Three commitments run through every page, and they are not decoration. They are the spec the whole book is checked against:

- **Privacy first.** The end state is your own machine answering your own questions. Nothing on the core path requires handing your data to anyone.
- **Open source as high as the stack goes.** Every software layer you install and run is open: you can read, change, and rebuild it. The honest limit, taken up just below and again in the appendix, is the firmware and driver floor beneath that software, which is not all open today and which no home can fully audit yet.
- **No external vendor reliance.** Nothing you rent does your thinking or holds your data: no cloud subscriptions, no SaaS, no partner lock-in. The honest exceptions are utilities and atoms, not services: electricity, an internet connection to carry traffic, a mobile plan if you want reach from anywhere, and hardware from a shop that can post you a replacement part. You rent dumb transport and raw power, never intelligence and never custody of your data, and because encryption makes that transport blind (Part 6.5), even the pipe you rent cannot read what crosses it.

An honest qualification, because this book does not trade in absolutes. “Depending on nobody” is the direction of travel, not a place you can fully arrive. Beneath the open software you install sits a floor you did not write and cannot fully read, and it is far wider than the one processor everyone pictures: a modern machine is dozens of chips, not one, and almost each carries its own firmware or microcode, the disk and solid-state controllers, the network interface, the graphics card, the keyboard and the embedded controllers on the board, the power and management circuitry, nearly none of it open. Some of the software riding just above that floor is closed too: the fastest consumer path to local AI today runs on graphics cards whose drivers are still largely proprietary, an honest dependency this book leans on and names rather than hides. A complete map of every piece of code executing on your own machine is, in the current ecosystem, simply not obtainable, and pretending otherwise would be exactly the kind of overclaim this book refuses. Trust assumptions live in that floor, and at the very bottom they cannot be removed, only acknowledged, and narrowed where you can: opening and mapping that layer is precisely the standing goal an AI-native, security-and-privacy-first ecosystem should be built toward, even though no one has reached it. So sovereignty is not the fantasy of trusting no one. It is the discipline of depending on as few as possible, knowing exactly who they are, mapping as much of the rest as you can, and arranging things so that a betrayal anywhere is survivable.

That last part is what network security actually is, properly understood. You cannot verify every transistor, so instead you isolate every component, service, and store of data thoroughly enough that none of them has to be trusted blindly: each one is watched, its communication is expected to stay within narrow bounds, and anything outside those bounds is spotted, flagged, and answered, with the response escalating as other signals agree something is wrong. Done well, the result is a system whose safety does not rest on any single part being trustworthy, but on the whole watching the moment any part stops behaving. The unavoidable trust at the bottom stops mattering because nothing is left unwatched above it, and Parts 7 and 8 build exactly that, piece by piece.

Some people call this “web4”: a local-first layer that sits above the public internet and treats that internet as somewhere you reach out to by choice, rather than somewhere you live inside. Keep all the public web you want. The difference is that it becomes opt-in, on your terms, in your formats, at your frequency. Your center of gravity moves home.

A clarification that this whole volume earns, so hold it lightly until Part 6 pays it off: the internet is not fragile. It is one of the most robust systems humans have built. The argument here is not fear of the network failing. It is that *not needing* the network is a different and underrated kind of power: it gives you privacy, speed, resilience, and ownership all at once, and it turns the WAN (Wide Area Network) from a leash into a tool you pick up when you want it. That benefit, the value of WAN-optional rather than WAN-dependent, is a thread we follow to its conclusion in Part 6.9.

A day with a home node

To make the abstract concrete before the building begins, picture an ordinary day once the node is running. You wake to a digest your node composed overnight from sources you chose, summarised by your own model, your interests never sent anywhere. Over breakfast you ask it something half-remembered from a book and it answers from your offline library, no signal required. On the train you open your voice app from your phone, record a thought, and it transcribes on the box at home, your voice touching no one else’s server. At work a question about something obscure sends your node into its offline Wikipedia and back with a grounded answer. That evening a drive in the node reports it is tiring, and a reorder is already drafted in your inbox waiting for one click. You pull the internet cable to move the

router, and nothing you rely on even notices, because none of it was ever out there. Late at night you change a small thing about how the node works, run the tests, watch them pass, and trust it.

None of that is science fiction and none of it rents anything. It is one person, one machine, and a set of principles applied. Every piece of that day is built in the parts that follow, and the thread running through all of it is the same: your tools work for you, on your ground, whether or not the wider world is reachable at that moment. That is what owning your ground feels like in practice, and it is what this volume hands you.

THE COMPANION APP, AND WHY THIS IS ITS MANUAL

This book does not come alone. It ships beside a real, fully open-source application: an AI-native voice-transcription app, built end to end with the exact principles in these pages. The voice app in the day-with-a-home-node vignette is not a thought experiment. It is the app, and you can run it.

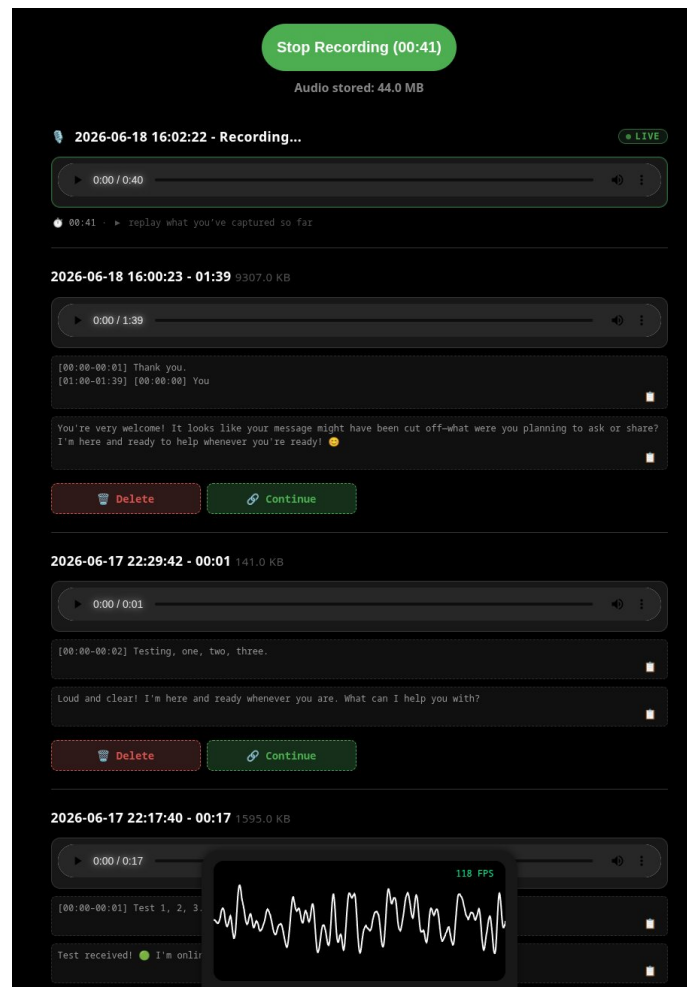
What it does is the drawer model (Part 0.3) made concrete. It is a progressive web app (PWA, a website that behaves like an installed app), so it runs in any browser with nothing to install. It records audio on your device, plays that audio back live while you are still recording, and transcribes it using the local speech service on your own node, so the recording and the transcript live on your device and your node and touch no one else's computer. And because it was built the way Part 9 teaches, it arrives with its own suite of tests, the executable definition of what "working" means for it.

The relationship between the two is the whole point: the code is the worked example this book builds toward, and the book is the manual the code needs. The app shows you a finished local-first application; the book explains every layer beneath it, how to run it, and, above all, how to make it fully functional safely. They go hand in hand. Read the book to understand the app; run the app to see the book made real.

This is also why installation cannot be a single script you run and forget. Getting the app merely running on your own machine is nearly that simple. But unlocking its full reach, the ability to open your voice notes from your phone on the far side of the planet, requires changing your router to forward a port, and that change crosses you from your private LAN onto the public WAN (Part 0.2). A script can flip that setting in a second. What a script cannot do is give you the judgement to know whether you should, and what you are exposing when you do. Before you make your node reachable from the open internet, you should understand exactly what the open internet does to an exposed door (Part 7.9) and how to defend it (all of Part 7). So this book deliberately walks you up to the router change with your eyes open, teaching the security first and flipping nothing on your behalf, because the entire premise of a sovereign stack is that you, not a script and not a vendor, are the one who decides what your machine exposes. The manual is longer than a script precisely because understanding is the product, and the code is what understanding lets you safely switch on.

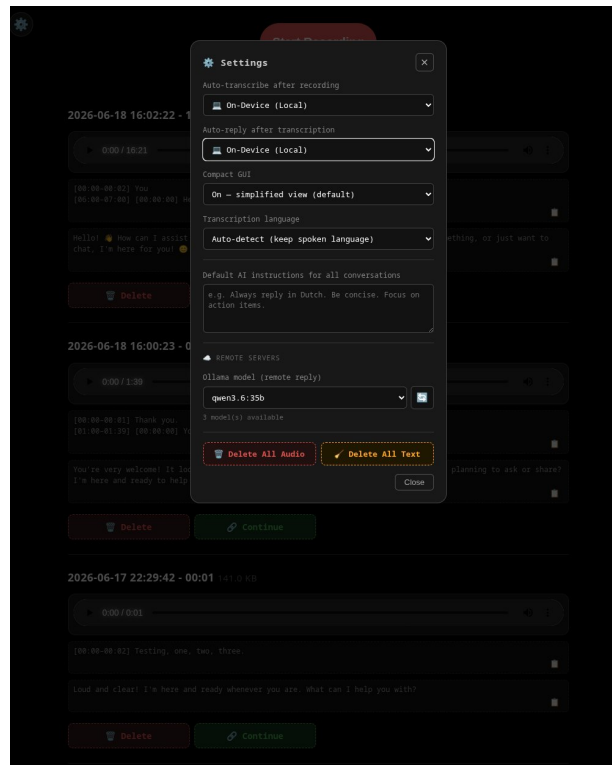
What it looks like, and what it does

Words describe it, but a picture answers "what am I actually getting" faster. Here is the app in use, and then the same thing as a plain feature list, so you know what to expect without reading its test suite.



The main view: record, replay live, transcribe, and let your model reply, all on your own node.

The app records in your browser with a live timer and a running tally of how much audio you have stored, draws a moving waveform while you speak, and lets you replay what you have captured so far without stopping the recording. Each take becomes a dated entry with its own player and a timestamped transcript, and your model replies to that transcript right underneath it, turning a voice note into a back-and-forth you can continue across turns or delete. Everything it can be told to do lives in one settings panel.

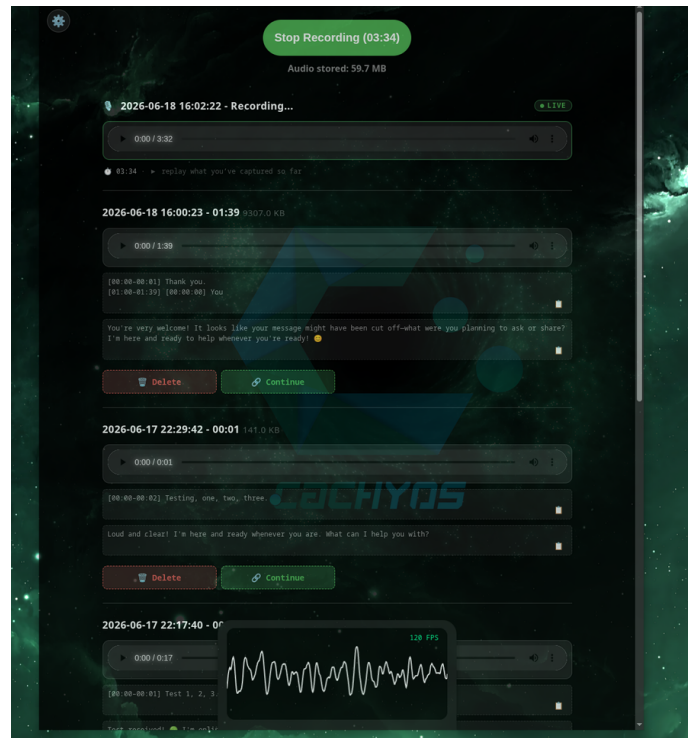


Settings: on-device by default, with remote servers as an explicit opt-in.

In plain words, the current feature list is this:

- **Record, with feedback.** A single record control with a live elapsed timer, a running “audio stored” size, and a real-time waveform while you speak.
- **Replay while recording.** Hear what you have captured so far without stopping, the play-while-recording behaviour built and tested as its own feature.
- **Transcribe on-device.** Each recording is transcribed locally by default, with per-segment timestamps, so the audio never has to leave your node.
- **Let your model reply.** After transcription, your local model answers the content, so a spoken note becomes a conversation rather than just a transcript.
- **Keep or clear.** Every entry has a player, a copy button, and delete, and bulk “delete all audio” and “delete all text” are one click each.
- **Choose local or remote, per step.** Transcription and reply can each be set to on-device or to a remote server independently, with on-device the default and remote an explicit choice.
- **Set a standing instruction.** A “default AI instructions” field acts as a system prompt for every conversation (for example, “always reply in Dutch, be concise, focus on action items”).
- **Pick your language and your view.** Transcription language can auto-detect or be fixed, and a compact view simplifies the interface.

That whole list is the executable definition of the app turned back into English. The tests in the code say the same thing in a form a machine can check (Part 9); this is the form a person can read before deciding to run it. And here is that whole list in motion: the app running on a real node, mid-conversation, a live take recording at the top and earlier takes stacked beneath it, each with its own transcript and the node’s reply.



The app on a running node, mid-session: a live take up top, earlier takes below, each with its own player, a timestamped transcript, and your node's reply, ready to continue or clear.

Two apps, and a pattern. The node actually arrives with two open-source AI-native apps, and they could hardly be more different. The first is the voice transcriber you have just seen, which this book explains in full because it is the perfect size to learn from. The second has no consumer face at all: it is a small research lab that grows a network from a single rule and measures the shape of the space that emerges, asking whether something as rich as our four-dimensional world can arise from the smallest possible set of rules. The rule it ships with does not produce that, and that was never the point. The point was to ask what structure the simplest rules can grow, and to make asking it something a machine on your own shelf can do.

Both are the same shape underneath, the shape you will meet throughout this book: a small service on a local socket, reached only through the one front door, with your LLLM as the connective tissue. That shape is general, which is why the same node could just as easily host a file browser for your folders, a scheduler that starts jobs at set times (the agents of Part 8.6 are exactly this, your computer's own calendar of work), or anything else you can describe and test. Volume 1 explains the simple app in full and includes the research lab as it stands, because one worked example you understand completely teaches more than a shelf of half-sampled ones. The pattern is the gift. The two apps are just the first two things you point it at, and the appendix sketches where the second one leads, which is somewhere genuinely strange.

PART 0: THE MAP

Mental models, before a single command.

0.1 The abstraction ladder

Everything you will touch is a stack of layers, and each layer's only job is to hide the one beneath it:

```
electrons moving through silicon
-> logic gates (AND, OR, NOT)
  -> machine code the CPU executes
    -> a programming language (Python, JavaScript)
      -> functions
        -> modules and classes
          -> services talking over a network
            -> the app you actually use
```

You do not need to master every rung. You need to know the ladder exists and to always know which rung you are standing on. Because the real daily work, debugging, is almost always one question: which layer is lying to me? Is the app wrong, or the service it calls, or the network between them, or the operating system, or the disk? Whoever can place the failure on the correct rung fixes it. Whoever cannot, flails.

Here is the move in practice. Your web app shows an error. Before changing any code, you ask which rung. Is the browser even reaching the server (network rung)? Is the server running at all (service rung)? Did the server get the request but mishandle it (application rung)? Did the application ask the disk for something missing (storage rung)? Each rung has a way to check it, and you walk down until a rung answers wrong. That rung is the bug. This single habit, descend the ladder until a layer misbehaves, is the most valuable debugging skill there is, and it is why the rest of this book keeps naming which rung you are on.

Pointer. When you hit an error you do not understand, paste it into your model with this framing: "Here is an error from my system. I want to debug by layers. List the layers involved in this request from my browser down to the disk, and for each one give me a single command or check I can run to tell whether that layer is the problem." You are not asking for the answer. You are asking for the ladder, so you can find the answer yourself.

There is a deeper symmetry in the ladder worth carrying. Math is pure relation, structure with no substrate. Numbers are math pinned down, made discrete and countable. Technology is number forced into matter, where holding a single state suddenly costs energy. And a program is technology handed instructions: math that runs, and therefore math that can fail. That last word is why this book has a whole part on verification and a whole part on security. Pure math never breaks. The moment you make it run on a machine reachable by the world, it can.

0.2 LAN versus WAN: the most important distinction in this book

Read this section twice.

Your **LAN** (Local Area Network) is the network inside your home: your devices, your WiFi, your Ethernet, all sitting behind one box, your router. Inside the LAN, devices talk to each other directly using private IP (Internet Protocol) addresses, addresses reserved by global agreement to mean “local only” and to never appear on the public internet:

- 10.0.0.0 to 10.255.255.255
- 172.16.0.0 to 172.31.255.255
- 192.168.0.0 to 192.168.255.255

If a device’s address starts with 192.168. or 10., it is speaking on your LAN.

Your **WAN** (Wide Area Network) is everything outside, the public internet. Your router has one public-facing address (your public IP), handed to you by your internet provider, and it acts as the single guarded door between your private LAN and the wild WAN.

This distinction governs everything, because the threat model on each side is completely different.

On the WAN, you must assume every packet can be seen and tampered with by strangers, providers, and whoever sits on the wires between you and the far end. That is exactly and only why HTTPS exists: to encrypt traffic so that crossing hostile ground reveals nothing and changes nothing.

On a LAN you physically control, your own home, your own cable, your own WiFi with a strong password, the ground between two of your devices is not hostile in the same way. A request from your laptop to your home node never leaves your house. This is why HTTPS is mandatory over the WAN but genuinely optional on a trusted, physically secured LAN. Talking to your node at `http://192.168.1.50:3000` from your own couch is fine. Talking to it at `http://your-public-ip:3000` from a cafe is reckless.

The caveats are the whole craft, so be precise:

- “Trusted LAN” means you control the physical layer and the access. A shared dorm network, an office LAN, public WiFi, a network with guests or untrusted smart-home gadgets on it: those are not trusted. Treat them like the WAN.
- WiFi is radio. “Physically secured” is harder for WiFi than for a cable you can see and trace, because the signal leaks past your walls. A strong WPA (WiFi Protected Access) password is your fence. Without it, your LAN is broadcasting to the street.
- The instant you want to reach your node from outside the house, you have crossed onto the WAN, and HTTPS stops being optional. That transition is the subject of Part 6.

Hold this the whole way through: inside is a different country than outside, and the border is your router.

Pointer. Ask your model to quiz you: “Give me five short scenarios (a laptop at home, a phone on cellular, a guest’s phone on my WiFi, a friend across the country, my own server in another room) and for each one ask me whether the connection is LAN or WAN and whether HTTPS is required. Then tell me if I got each right and why.” Testing yourself on this one distinction pays off more than any other single thing in the book.

0.3 Two security models: the vault and the drawer

Most software you have used follows the **vault** model: a central server holds everyone's data, so the whole security budget pours into defending that center. It is a single juicy target, and one breach exposes everybody. It demands constant guarding, retention policies, breach-notification machinery, and trust that the operator never errs.

This book mostly builds the other model: the **drawer**. You relocate the trust boundary out to the edge. Each device holds only its own data, locally. The server in the middle is demoted to a stateless relay, custodian of nothing. The center stops being worth attacking, not because you fortified it, but because you emptied it. You cannot leak what you never held. There is no "did we accidentally log personal data," because the honest answer to "what do you store about me" is nothing, go look.

This is the same shift behind end-to-end encryption and local-first software, and it changes the whole risk sheet. Confidentiality at the center mostly vanishes. What remains is availability (a reachable door can always be knocked on) and client-side custody (the data now lives on each device, inheriting that device's safety and its fragility). The drawer model does not delete risk, it transfers it: the user gains sovereignty over their data and, with it, the responsibility for it. We will meet both faces of that trade repeatedly, and the voice recorder you serve in Part 6 is a working example of the drawer model: the audio and the transcript live in the browser on the device, and the node in the middle stores nothing.

0.4 What "running a server" actually means

Strip the mystique: a server is just a program that listens at an address and answers requests. The machine it runs on is incidental. A laptop can be a server. A desktop can be a server. The same service runs identically on both. What differs between machines is how much work they can do at once and how well they tolerate running all day and night. So when this book says "your node," picture any computer you have designated to sit there and serve. Choosing which computer is Part 1.

A useful first instinct: you have almost certainly already run a server without calling it that. Any time a tool said "open your browser to localhost," a program on your own machine was listening at an address and answering your browser. That is a server. The leap in this book is not learning a new and mystical thing. It is taking that familiar local program, making it run reliably all day, giving it a real job (an AI, a web app, an agent), and deciding, deliberately and safely, who else is allowed to reach it.

0.5 Files, processes, and ports: the three things you will always check

Almost every problem on a node is one of three things, and knowing which collapses most debugging into a single quick check. A **file** is in the wrong place, missing, or unreadable (wrong path, wrong permissions). A **process** is not running, has crashed, or is the wrong one (the service died, or two copies are fighting over the same resource). A **port** is closed, bound to the wrong interface, or blocked (nothing is listening, it is listening on localhost only, or a firewall is in the way). When something breaks, ask which of the three it is, and check it directly:

```
ls -la /path/to/thing      # FILE: is it there, can my user read it?
systemctl status myservice # PROCESS: is it running, or did it crash?
sudo ss -tlnp | grep PORT  # PORT: what is listening, on which interface?
```

You will run these three lookups more than any others on your node. They are the practical face of the abstraction ladder from 0.1: a request fails, and the cause is almost always a file the program could not read, a process that was not running, or a port that nothing answered on. Get fluent in these three checks and you have the daily reality of operating a node mostly handled, because you can always find out, fast, which of the three is wrong.

0.6 A worked debugging story, start to finish

Make it concrete, because the method matters more than any single command. You open your app in a browser and get nothing at all. Walk the ladder, one rung at a time, changing nothing until a layer misbehaves.

First, is the browser reaching anything? From another machine: `curl -v http://NODE_IP↵:3000/`. It refuses instantly. A refusal (rather than a hang) means something actively said no or nothing is listening, which points below the application layer.

Second, is the process even running? On the node: `systemctl status myapp`. It shows “failed.” You have found a rung.

Third, why did it fail? `journalctl -u myapp -n 50`. The log says it could not bind to port 3000 because the address is already in use. Now it is a port and process conflict, not a code bug.

Fourth, who is holding the port? `sudo ss -tlnp | grep 3000`. An old copy of the app, left over from last night, is still running and squatting on the port.

Fifth, fix and verify. Stop the stray process, restart the service cleanly, and `curl` again. The page loads.

Notice what did not happen. You never guessed, and you never edited a single line of code. You descended the ladder one rung at a time, let each layer hand you the truth, and the bug revealed itself as a stale process holding a port. That is the entire skill, and it is the same motion whether the cause turns out to be a file, a process, or a port. The person who can do this calmly does not need a tutorial for every problem, which is the whole point of the book.

Pointer. Teach your model to debug *with* you rather than *for* you: “Something is broken on my node. Do not guess the fix. Instead, walk me down the layers from my browser to the disk, and at each layer give me one command to run and tell me what a healthy result versus a broken result looks like. I will run them and report back what I see, and we will narrow it down together.” This keeps you in the driver’s seat and builds the instinct you will eventually use without any model at all.

PART 1: THE HARDWARE, AND THE MEMORY-BANDWIDTH LENS

1.1 The only question that matters: what will it run?

A home node for local AI has one demanding tenant, the AI model, and everything else (the proxy, the agents, the web app) is featherweight beside it. So size the machine to the model, then know the rest comes nearly free.

Large language models are memory-bound. To run one, the model's weights must sit in fast memory while it works. There are two kinds of fast memory, and the distinction is the single most important hardware fact in this book:

- **System RAM** (Random Access Memory): plentiful and cheap, used by the CPU (Central Processing Unit). A model can run here, but slowly.
- **VRAM** (Video RAM): the dedicated memory on a GPU (Graphics Processing Unit). A model that fits in VRAM runs much faster, because GPUs are built for the parallel math neural networks are made of.

A third arrangement, **unified memory** (Apple silicon, and newer AMD and NVIDIA mini-PC chips), shares one fast pool between CPU and GPU, which is unusually good for running large models without a giant discrete card.

1.2 Memory bandwidth is the lens

Here is the idea that makes the rest of this part, and the budget in Part 5, legible.

When a model generates text, it produces one token at a time (a token is roughly a word-piece), and to produce each token it must read the model's entire set of weights out of memory. So the speed of generation is governed, more than by anything else, by how fast you can read memory. That number is **memory bandwidth**, measured in gigabytes per second (GB/s).

This gives a rule of thumb accurate enough to plan a purchase by:

tokens per second is roughly (memory bandwidth in GB/s) divided by (model size in GB).

A model quantized to 4 bits (the standard way to shrink weights with little quality loss) is, in practice, a little more than half a byte per parameter once the real formats are counted: a pure four-bit weight is exactly half a byte, but usable quantizations keep some layers at higher precision and add small per-block scales, which lands around 0.6 bytes per parameter, so an 8-billion-parameter model is roughly 4.8 GB. On a card with about 936 GB/s of bandwidth, the ceiling is about 936 divided by 4.8, near 195 tokens per second, and real-world results land around half that once overheads are counted, somewhere in the 95 to 112 range. The arithmetic predicts the measurement to within a comfortable tolerance, which is all you need. That is the whole lens: when you read a bandwidth number, you are reading a speed.

Two honest qualifications keep this lens from being mistaken for a precise instrument, and both matter. First, every tokens-per-second figure in this book is an approximation, a window

and not a point: the real number also depends on the runner, the exact quantization, the operating system's overhead, and whether the machine is staying cool, so treat any such figure as a rough order of magnitude to decide by, not a guarantee. Second, the speed is not even constant for one model on one machine: as a conversation or a document grows, the context the model must carry grows with it, the running text and the key-value cache that holds it, and generation slows as that context fills, so the same model that begins a fresh chat briskly will answer noticeably slower deep into a long one. Read the formula as the ceiling on a nearly empty context; expect to live a little below it in practice, and lower still as the window fills up.

Pointer. Hand your model your situation and let it do the arithmetic: "I am looking at a GPU with about X GB/s of memory bandwidth and Y GB of VRAM. Using the rule that tokens per second is roughly bandwidth divided by model size, and assuming a real-world efficiency around one half, estimate how fast it would generate an 8B, a 14B, and a 32B model at 4-bit, and tell me which of those even fit in Y GB with room for context. Show your working so I can sanity-check it." Now you can evaluate any card you find for sale in under a minute.

1.3 The bandwidth timeline: how technology lined up

Lay one ruler, memory bandwidth, across eras and devices, and the progression of computing becomes a single axis. Treat every figure below as approximate, a window rather than a point. They are here to show shape and scale, not to be quoted to the decimal.

Device or part (era)	Memory bandwidth	Memory	Note
Original iPhone (2007)	on the order of 1 GB/s	128 MB	Historical floor, rough
Typical laptop, dual-channel DDR5-5600 (2024 to 2026)	about 90 GB/s	16 to 64 GB	Simple arithmetic: 5600 x 8 x 2
Apple M4 (2024)	about 120 GB/s	up to 32 GB	Unified
Apple M4 Pro (2024)	about 273 GB/s	up to 64 GB	Unified
AMD and NVIDIA mini-PC unified chips (2026)	about 256 to 300 GB/s	up to 128 GB	LPDDR5X unified
Apple M4 Max (2024)	about 410 to 546 GB/s	up to 128 GB	Unified
Apple M-series Ultra (M1 to M3)	about 800 to 819 GB/s	up to 192 to 256 GB	Two dies fused
RTX 3090 (2020)	about 936 GB/s	24 GB	GDDR6X, the value pick
RTX 4090 (2022)	about 1,008 GB/s	24 GB	GDDR6X
RTX 5090 (Jan 2025)	about 1,792 GB/s	32 GB	GDDR7, fastest consumer card
A100 (2020, data center)	about 1,555 GB/s (40 GB) to 2,039 GB/s (80 GB)	40 to 80 GB	HBM2 and HBM2e
H100 SXM (2022, data center)	about 3,350 GB/s	80 GB	HBM3
H200 (2024, data center)	about 4,800 GB/s	141 GB	HBM3e
B200 (2025, data center)	about 8,000 GB/s	192 GB	HBM3e
Vera Rubin (roadmap, H2 2026)	about 13,000 GB/s	288 GB	HBM4

A note on the memory-type names in the table, for the curious: GDDR (Graphics Double Data Rate) is the fast memory on a consumer graphics card, HBM (High Bandwidth Memory) is the even faster memory stacked on data-center accelerators, and DDR (Double Data Rate) is the ordinary system memory in a typical computer. You do not need to track the differences; they are all just memory, and the only number that matters for the lens is the bandwidth.

Two things jump out. First, the home-affordable parts (the 3090 through the 5090, and the Apple unified chips) sit in the 500 to 1,800 GB/s band, which is the same order of magnitude as a 2020 data-center A100. A used card from 2020 on your desk is not a toy. Second, the data-center monsters (H200, B200) are roughly two to five times faster again, but as Part 5 shows, that extra speed buys a home user almost nothing, because a home user is one person asking one question at a time. The ruler makes the budget obvious.

It is worth pausing on what this ruler measures across history. The original iPhone in your pocket in 2007 moved memory on the order of a gigabyte per second. A cheap laptop today moves around ninety times that, and a used graphics card from 2020 moves nearly a thousand times that. The reason a private, capable AI at home became possible in this decade and not the last is mostly this one curve. The models got better, yes, but the thing that crossed the line from “lab only” to “on a shelf in your house” was memory bandwidth becoming cheap. You are building on the first generation where that is true.

1.4 RAM versus VRAM versus unified memory

The practical hierarchy follows straight from the lens. A model that fits in VRAM runs fast (highest bandwidth). A model that fits only in system RAM runs on the CPU, slowly (system RAM is roughly ten times slower than a GPU’s VRAM). A model that fits in neither will not run usefully at all: it spills to disk and crawls, often five to ten times slower than fitting in memory. So the first question for any model is not “is my CPU fast” but “does it fit in my fastest memory, with room to spare for the operating system and the growing context of a long conversation.”

Unified-memory machines are the quiet outlier. An Apple M4 Max with 128 GB of unified memory holds a 70-billion-parameter model entirely in fast memory, which no 24 GB or 32 GB consumer GPU can do, and it does so drawing roughly 60 to 90 watts for the whole system, several times more power-efficient than a discrete card. It trades absolute speed (its bandwidth is about a third of a 5090’s, so it generates tokens proportionally slower, around 55 per second on an 8B model versus the 5090’s roughly 186) for the ability to keep very large models resident without splitting them across cards. For a home that wants big models in near silence, that trade is often the right one.

One honest note while you are choosing, consistent with the openness the rest of the book argues for: the fastest consumer path to local AI today runs on NVIDIA cards, whose drivers are still largely proprietary (an open kernel module now exists, but the full stack is not open), and much of the firmware on any card you buy is closed regardless of brand. None of this stops you owning and running the machine, but it is a real closed-software dependency at the hardware layer, named here rather than glossed over, and the thesis in Part 0 and the appendix both return to why that floor matters and where it ought to head.

1.5 Choosing a chassis: mini-PC versus desktop versus laptop

All three can be your node. They trade differently.

Mini-PC (a small pre-assembled “desktop in a lunchbox”). The natural default for a home node. Low power draw (often 10 to 35 watts idle), silent or near-silent, small enough to tuck on a shelf, happy running all day. The catch is that most have integrated or unified graphics rather than a powerful discrete GPU, so they shine on small-to-mid models and tap out on the largest. The newer unified-memory mini-PCs and the Apple Mac mini are notable because their shared memory punches above their size for AI. Best when you want an always-on, quiet, efficient node and small-to-mid models are enough.

Desktop (tower). The most capacity per euro and the only easy path to a big discrete GPU

with lots of VRAM, plus room to add memory, storage, and cooling later. Louder, hungrier for power, bigger. Best when you want serious model sizes or raw speed, and you can give it a corner.

Laptop. It works, and its battery is a free uninterruptible power supply that rides through brief outages, which (foreshadowing Part 6.9) is a quietly large benefit for a node that should not care about a flickering mains supply. But laptops are thermally and electrically constrained: they throttle under sustained load, hold less memory, and rarely carry a high-VRAM GPU. Running one pinned at full load all day also ages the battery. Best when you are starting with hardware you already own, or you value portability over capacity.

The reusable criteria, in priority order for local AI: first, fast memory (how much, and is any of it VRAM or unified); second, sustained thermals (can it run hot for hours without throttling); third, power and noise (it lives in your home, on, always); fourth, expandability (can you add memory, GPU, storage later). Core count and clock matter, but less than memory for this workload.

Out of scope, deliberately: rack-mounted server-class hardware. It is wonderful and entirely unnecessary for a first home node, and it brings noise, heat, power bills, and complexity that work against the goal. Everything in this book runs beautifully on a quiet box on a shelf.

Pointer. Turn your model into a buying advisor with full context: “I want an always-on home node for local AI. My budget is X euros, I care about [silence / speed / running the biggest models / lowest power]. I have these constraints: [space, noise, whether I already own a machine]. Walk me through the trade between a mini-PC, a desktop with a used 24 GB GPU, and a unified-memory machine for my case, and tell me what model sizes each would run at what rough speed. Ask me clarifying questions before recommending.” The point is not to be told what to buy. It is to be walked through the reasoning until the choice is obviously yours.

1.6 A worked purchase: from “I want this” to a parts list

Make the lens concrete with one full example, the kind of reasoning you will repeat. Suppose you decide you want to comfortably run models up to about 32 billion parameters at interactive speed, keep an offline Wikipedia, and stay near a middle budget.

Step one, the model sets the memory floor. A 32B model at 4-bit is roughly 18 to 20 GB, and you want headroom for the operating system and a long conversation’s growing context, so you need about 24 GB of fast memory. That points straight at a 24 GB card.

Step two, the lens sets the speed expectation. A 24 GB card in the roughly 900 to 1,000 GB/s band will generate an 8B model near 100 tokens per second and a 32B near the 30s, which is faster than you can read. Good enough, by a wide margin.

Step three, the rest of the box is cheap by comparison. Around the card you need a host: a modest CPU, 32 to 64 GB of system RAM (so the machine is comfortable even though the model lives in VRAM), a power supply sized for the card with margin, a case, and an SSD (Solid State Drive) large enough for your models plus an offline Wikipedia (Part 5 shows that is a couple hundred gigabytes, so a 2 TB drive is lavish).

Step four, the storage doubles as your retention plan. Pick the drive one size larger than you think you need, because Part 7’s backups and Part 8’s retention of snapshots both want room, and storage is the cheapest thing in the build.

That is the entire method: the model you want sets the memory, the memory sets the card, the lens sets your speed expectation, and everything else is a comparatively cheap host wrapped around it. Re-run those four steps with different targets and you can spec any tier in minutes.

1.7 Storage, and the one boring essential

Use an SSD, NVMe (Non-Volatile Memory express) if the machine supports it, because models are large files you load often and a spinning disk makes everything feel broken. Size it for your models plus comfortable room. As Part 5 shows, a complete offline Wikipedia with its embeddings is on the order of a couple hundred gigabytes, so even a 1 TB drive holds it with space to spare, and a 2 to 4 TB drive gives lavish room for retention and backups.

And the boring essential the whole local-first philosophy demands: a backup plan, because there is no cloud silently keeping a copy for you anymore. That is Part 7, and it is not optional.

1.8 Power, heat, and what it costs to run all year

A node is a computer that never turns off, so its running cost is a real number worth knowing before you buy. The two figures that matter are idle draw and load draw. A quiet mini-PC might idle near 10 to 20 watts and a desktop with a big GPU near 60 to 100 watts, while under heavy AI load that desktop can briefly pull several hundred watts as the GPU works. Because a home node sits idle most of the time and only spikes when you actually ask it something, your yearly cost is dominated by the idle figure, not the peak. The arithmetic is simple: watts, times hours in a year (about 8,760), divided by 1,000, gives kilowatt-hours, times your electricity price gives the annual cost. A 20-watt node costs roughly 175 kilowatt-hours a year; a 60-watt one, three times that. This is why “quiet and efficient” is not just about noise, it is about the bill, and it is a strong argument for a mini-PC or unified-memory box if your model sizes allow it.

Heat follows power: every watt a node draws becomes heat it must shed, which is why sustained thermals from 1.5 matter. A node that runs hot all summer in a closed cabinet will throttle (slow itself to avoid damage) or age faster. Give it airflow. A shelf with space around it beats a sealed drawer.

And a small, genuinely useful piece of resilience that Part 6.9 will return to: an uninterruptible power supply (UPS), a battery that sits between the wall and your node, rides through brief mains flickers and gives the node time to shut down cleanly during a longer outage. A laptop has this built in, which is one of its quiet advantages as a node. For a desktop, a modest UPS turns a power blip from a crash into a non-event, which matters when your node is the thing your house relies on.

1.9 Buying used safely

The value tiers in this book lean on the used market (a 2020-era 24 GB card is the budget hero), so a few words on buying secondhand without getting burned. Used graphics cards are generally safe to buy because they have few moving parts, but vet them: ask how the card was used (gaming is gentler than nonstop mining, though a well-cooled mining card can be fine), ask for a photo of the actual card with a timestamp, and prefer sellers who will let you test it. When it arrives, test it before you trust it: confirm it shows up at expected clocks, run a memory check to catch bad VRAM, and run a real, sustained AI workload for an hour while watching temperatures and for any garbled output, which is the rare sign of damaged memory. Be aware that some manufacturer warranties are dead (a few brands have exited the market or expired their coverage), so on the used market you are usually buying without a warranty and the price should reflect that.

The principle underneath: the used market is where the bandwidth-per-euro is best, because last generation’s high-end card is this generation’s bargain, and for AI inference (where capacity and bandwidth matter more than the newest features) that bargain is exactly what you want. Buy the criteria, not the badge: enough fast memory to hold your target model, enough

bandwidth to generate it at a speed you can read, and a seller you can verify.

Pointer. Before committing to a used part, have your model build you a checklist: “I am buying a used [specific GPU or machine] for local AI. Give me a pre-purchase checklist (questions to ask the seller, red flags in the listing) and a post-arrival test plan (commands to confirm it is healthy, what temperatures and behaviours are normal versus alarming) so I can verify it works before I trust it.” A ten-minute test on arrival is cheaper than a regret.

PART 2: THE OPERATING SYSTEM, LINUX AS HOME BASE

2.1 Why Linux, and the one law of this book

A node runs Linux because Linux is free, runs for years without nagging, is built to be operated remotely over a network, and is the native home of every server tool you will use. For the base, this book takes Arch Linux as its reference, for one reason that sits at the heart of Part 7: minimalism is a security tactic. Every package you install is one more piece of software you are trusting and one more sliver of attack surface, so the fewer of them sit between you and the bare machine, the less there is to break and the less there is to attack. Arch installs almost nothing you did not ask for, which is exactly the property you want under a node you intend to defend, and it is more honest about what is actually on the machine than a distribution that arrives pre-filled with conveniences. No mainstream family makes “ship only what you chose” as much its default as Arch does, which is why it is the natural base here. The trade is that Arch is a rolling release (a continuous stream of current packages rather than a frozen snapshot patched for years), so you stay current by habit, which is the cheapest security there is (2.4), and you lean a little harder on the recovery mindset (2.8) and your backups (Part 7) for the occasional update that wants attention. The honest contrast is the long-term support (LTS) model that Debian and Ubuntu follow, which freezes versions and trades minimalism for years of rare, well-tested change; that is a reasonable choice too, and if you already live in that family your model will translate every command in this book to it. This book simply picks the smaller attack surface.

One worry to set aside at once, because it decides who this book is for: choosing Arch does not mean assembling an operating system by hand. If you are determined, you can install bare Arch from scratch, and it is a superb way to learn what is truly on a machine, since you add each piece deliberately. But most people should start from an Arch-derived, desktop-ready distribution that gives you the same minimal Arch base and the same pacman package manager behind a friendly installer and a working desktop out of the box. CachyOS and Manjaro are two well-known examples: ordinary, ready-to-use desktop systems that are Arch underneath, so everything this book shows works on them directly, and you can always trim back toward the minimal Arch core later if you choose. Begin with one of those and you get the minimalism philosophy without the from-scratch effort, which is exactly the accessible on-ramp this book wants for a first-time reader.

Now the law this whole book turns on, stated plainly here because Linux is where you first meet it: **no instruction is guaranteed to work on your system.** Your distribution version, your firmware, your hardware, your network, your country’s defaults: there are more variables than any author can enumerate, and a command that worked on the writer’s machine may not work on yours. This is not a flaw in the book. It is the nature of running software on real, varied machines.

So the durable skill this book actually teaches is not memorising commands. It is **learning to converse your way out of trouble.** When something does not work, you do not give up and you do not blame yourself. You ask. And here is the ladder of self-reliance, from depending on others to depending on no one, which doubles as the structure of everything that follows:

1. **Ask a cloud model** when you are stuck and do not know yet. It is fine to start dependent. Everyone does.
2. **Learn to search well.** Search engines now answer with AI by default, so asking a clear question is the skill. “Learn to google” is not a joke, it is rung two.
3. **Run your own local model**, an LLLM, for everyday asking, offline and private (Part 4). Now your questions stop leaving the house.
4. **Feed it your own documents** with basic retrieval (Part 5). Now it answers from your material.
5. **Feed it all of Wikipedia** with local retrieval (Part 5). Now you hold a private, offline, queryable copy of general human knowledge, and you need no one’s server to ask it anything.

The slogan that carries the whole book: learn to google, until you make your own google. Each rung maps to a later part. The destination is privacy: a model you own, answering without a vendor in the loop. Hold this when a command in this book does not work on your machine. The fix is not in the book. The fix is the conversation the book is teaching you to have.

Pointer. Before you run any command from anywhere, including this book, get in the habit of asking first: “Explain exactly what this command does, token by token, what it will change on my system, whether it is reversible, and what the most common way to get it wrong is. I am on [your distribution].” This one habit turns copy-paste from a risk into a lesson, and it is rung one and two of the ladder practised on every single step.

2.2 First contact: the shell

The shell (bash, or zsh) is a text conversation with the machine, and it is the one interface that exposes every layer of the ladder. A few load-bearing ideas: the filesystem is a single tree starting at / (root); your stuff lives under /home/you; system configuration lives under /etc; logs live under /var/log. Commands are programs (ls lists, cd moves, cat prints a file, nano edits one), and the output of one can pipe into another with |. You do not need fluency to start. You need to be unafraid of the prompt and willing to read the error it gives you, because an error message is a gift: it is the lower layer telling you exactly what it refused to do.

A short orientation you can actually run on your first day, each line safe and read-only:

```
whoami      # which user am I
pwd         # where am I in the tree
ls -la      # what is here, including hidden files
df -h       # how full are my disks
free -h     # how much memory is free
uname -a    # what kernel and architecture am I on
ip addr     # what are my network addresses
```

None of these change anything. They are how you look around. Run them, read the output, and ask your model to explain any line that puzzles you. That loop, look, read, ask, is the entire beginning of competence.

2.3 Users, root, and sudo

Linux separates the all-powerful root user from normal users on purpose. Do not live as root: one careless command as root can erase the machine. Create a normal user and grant it the

ability to elevate temporarily with sudo:

```
adduser you
usermod -aG sudo you
```

Now you runs day to day and prefixes the occasional privileged command with `sudo`. This single habit prevents a large fraction of self-inflicted disasters. The mental model: `root` is the master key to the whole building, and you do not carry the master key in your pocket where you might drop it. You leave it in a safe and take it out for the one door that needs it, which is what `sudo` is.

2.4 Updates: the cheapest security you can buy

sudo pacman -Syu

Run it on day one and regularly after; on a rolling release like Arch, staying current is simply part of operating the machine rather than an occasional event. Most real-world break-ins exploit known holes that a patch already fixed weeks earlier, so keeping current is unglamorous and is the highest-return security action there is. One honest rolling-release habit goes with it: apply updates deliberately rather than blindly, glancing at the distribution's news before a large update for the rare change that needs a manual step, which is the small, worthwhile price of always being current instead of years behind. The screenshot below is exactly this command at work.

```

sqlite-3.53.2-2 sudo-1.9.17.p2-6.1 syndication-6.27.0-1.1 syntax-highlighting-6.27.0-1.1
systemsettings-6.7.0-1.1 tesseract-5.5.2-1.1 tesseract-data-eng-2:4.1.0-5
tesseract-data-osd-2:4.1.0-5 util-linux-2.42.2-1 util-linux-libs-2.42.2-1
vapoursynth-77-1.1 vim-9.2.0670-1.1 vim-runtime-9.2.0670-1.1 xdg-desktop-portal-1.22.1-1.1
xdg-desktop-portal-kde-6.7.0-1.1 zix-0.8.2-1.1

Total Download Size: 2169,69 MiB
Total Installed Size: 5162,10 MiB
Net Upgrade Size: 70,92 MiB

:: Proceed with installation? [Y/n] Y
:: Retrieving packages...
ollama-cuda-0.30.10-1.1-x86_64_v4 38,5 MiB 6,49 MiB/s 01:51 [---co o o o o o o o o o o o o o o o o] 5%
libreoffice-fresh-26.2.4-2.1-x86_64_v4 31,6 MiB 7,17 MiB/s 00:17 [-----C o o o o o o o o o o o o o o] 20%
linux-cachyos-lts-6.18.35-1-x86_64_v4 29,6 MiB 4,92 MiB/s 00:24 [-----C o o o o o o o o o o o o o o] 19%
linux-cachyos-7.0.12-1-x86_64_v4 29,2 MiB 6,92 MiB/s 00:17 [-----C o o o o o o o o o o o o o o] 19%
brave-bin-1:1.91.175-1-x86_64 24,1 MiB 5,30 MiB/s 00:21 [-----co o o o o o o o o o o o o o o] 17%
signal-desktop-8.15.0-1-x86_64 11,5 MiB 2,70 MiB/s 00:34 [-----co o o o o o o o o o o o o o o] 11%
qt6-webeengine-6.11.1-4-x86_64 11,2 MiB 2,63 MiB/s 00:31 [-----co o o o o o o o o o o o o o o] 11%
firefox-152.0.1-1.1-x86_64_v4 16,9 MiB 1950 KiB/s 00:35 [-----C o o o o o o o o o o o o o o] 20%
linux-cachyos-lts-headers-6.18.35-1-x86_64_v4 13,4 MiB 2,48 MiB/s 00:18 [-----C o o o o o o o o o o o o o o] 22%
gcc15-15.3.0+ro.gd4b0e8df15be-2.1-x86_64_v4 11,5 MiB 1972 KiB/s 00:21 [-----C o o o o o o o o o o o o o o] 22%
Total ( 0/209) 215,9 MiB 42,3 MiB/s 00:46 [----c o o o o o o o o o o o o o o] 9%
```

A system update in progress on an Arch-based machine (here CachyOS, with a themed terminal): this is the `pacman -Syu` above at work, listing what will change and then downloading and installing it. One of the packages being fetched here is `ollama`, the local model runner from Part 4.

2.5 SSH: reaching the node safely

SSH (Secure Shell) is how you operate a headless node, an encrypted remote terminal. From your everyday computer:

```
ssh you@NODE IP
```

Passwords are the weak way in. Keys are the strong way: you hold a private key, the server holds your public key, and only your key opens the door. Set it up once, on your own machine:

```
ssh-keygen -t ed25519
ssh-copy-id you@NODE_IP
```

Then turn passwords off so brute-force guessing becomes impossible. Edit `/etc/ssh/sshd_config` on the node, set `PasswordAuthentication no` and `PermitRootLogin no`, and restart SSH with `sudo systemctl restart ssh`. Now the only way in is something you physically hold. Keep that key safe and backed up, because losing it locks you out too. Sovereignty cuts both ways: the same wall that keeps everyone else out keeps a careless you out, which is exactly why Part 7's backups include your keys.

The private-key idea is worth dwelling on because it returns in Part 6. A private key is a secret that never crosses the wire. You do not send it to prove who you are; you use it to answer a fresh challenge the server poses, which proves you hold the key without ever revealing it. That property, prove possession without disclosure, is the strongest kind of authentication there is, and it is why key-based SSH is effectively immune to the password guessing that hammers every server on the internet around the clock.

2.6 systemd: making things run and survive

A service that runs only while you are watching is a toy. **systemd** is Linux's way of saying "start this on boot, keep it running, restart it if it dies." You describe a service in a small text file at `/etc/systemd/system/myservice.service`:

```
[Unit]
Description=My service
After=network.target

[Service]
ExecStart=/usr/bin/node /home/you/app/server.js
Restart=always
User=you

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now myservice
sudo systemctl status myservice
journalctl -u myservice -f
```

`Restart=always` means a crash self-heals. That one line is "which patterns persist, which patterns fade" turned into configuration. You will wrap every long-running piece of your node in one of these: the AI runner, the speech service, the web app, every agent. By the end of this book you will have half a dozen of these files, and together they are what make the node a thing that simply stays up rather than a thing you babysit.

Pointer. When you want to turn any program into an always-on service, ask: "Write me a minimal systemd unit file that runs [this exact command], as user [you], restarts on failure, and starts on boot. Then give me the three commands to install it, check its status, and follow its logs, and explain what each line of the unit file does." Then read the explanation before

installing it, so you own the file rather than just pasting it.

2.7 Packages and the filesystem, a little deeper

Two pieces of Linux literacy pay off constantly. First, packages: software on Linux mostly comes from a package manager (pacman on Arch and its derivatives), which installs, updates, and removes programs along with everything they depend on. You rarely download installers from websites; you ask the package manager, which is both easier and safer because the packages are vetted and updated together. Other distribution families differ only in the tool, not the idea: the Debian and Ubuntu family uses apt, Fedora uses dnf, and each has the same verbs to install, update, remove, and search. This book uses pacman in its examples because Arch is its minimal reference base (2.1), and everything shown works directly on any Arch-derived distribution such as CachyOS or Manjaro; on a different family your model will translate any command to your own package manager in a moment, which is simply the cookbook method from the front of the book applied to the one place distributions genuinely diverge. The everyday verbs:

```
sudo pacman -Syu      # refresh package lists and upgrade everything
pacman -Ss keyword    # search for a package
sudo pacman -S thing   # install it (with its dependencies)
sudo pacman -Rns thing # remove it (and its now-unused dependencies)
pacman -Q              # list what is installed
```

Second, the filesystem has a logic worth internalising, because knowing where things live turns “where is that file” from a hunt into a lookup. Your data and configuration live under /home/you. System-wide configuration lives under /etc (the SSH config, the proxy config, systemd units). Logs live under /var/log. Installed programs live under /usr/bin and friends. Temporary and runtime state lives under /tmp and /var. Devices and disks appear under /dev and get mounted into the tree under /mnt or /media. You do not need to memorise all of it, but the instinct “configuration is in /etc, logs are in /var/log, my stuff is in /home” resolves most “where do I look” questions immediately.

2.8 When an update breaks something: the recovery mindset

It will happen eventually: an update changes behaviour, or a service that worked yesterday will not start today. This is not a catastrophe, it is a normal part of operating a machine, and the mindset matters more than any specific fix. First, do not panic and do not start changing many things at once, because that turns one problem into several. Second, find out what changed: `journalctl -u theservice -n 50` to read why it failed, and `cat /var/log/pacman.log` to see what was recently updated. Third, isolate: a service that fails to start almost always says why in its logs or in `systemctl status`, and that message, fed to your model, usually names the cause directly. Fourth, recover deliberately: most package managers let you reinstall or roll back a specific package, and most services can be restarted cleanly once the cause is fixed.

The deeper lesson is that this is exactly why Part 7’s backups exist and why the recovery mindset matters even more on a rolling release: a steady stream of current packages is the price of a small, current, minimal system, so backups and a known-good snapshot are what make any single bad update recoverable rather than fatal. A node you can always recover is a node you can update and experiment on without fear, which is the difference between owning a machine and being afraid of it.

Pointer. When an update breaks a service, hand the evidence to your model: “After updating, my service [name] will not start. Here is the output of `systemctl status [name]` and the last 30 lines of `journalctl -u [name]`. Tell me, step by step, what the most likely cause is, how to confirm it, and how to recover safely, including whether I should roll back a specific package. I am on [your distribution].” You are practising the recovery mindset with a guide, which is how you eventually do it alone.

PART 3: NETWORKING FROM ZERO

3.1 The request and response dance

The entire web is one motion repeated: a client opens a connection to a socket, sends a request, and gets a response. Pin the vocabulary. An IP address is a machine's location on a network. A port is a numbered door on that machine: web servers use 80 (HTTP, the HyperText Transfer Protocol) and 443 (HTTPS) by convention, your app might listen on 3000, a local AI runner on 11434. An address plus a port is a socket, the precise endpoint a request knocks on. DNS (Domain Name System) is the phone book that turns a human name into an IP number. When this book says "serving a webpage behind a port," it means exactly that: a program is listening at one socket, and when a request knocks, it answers.

You can watch the dance directly, which makes it real. From your node, asking your own machine for a page:

```
curl -v http://127.0.0.1:3000/
```

The `-v` (verbose) flag prints the whole conversation: the request your machine sends and the response it gets back, headers and all. Run it against a service you have started and you are watching the request-response dance at the lowest level you will ever need. Everything else in web serving is this, repeated and dressed up.

3.2 localhost versus 0.0.0.0: the bind that bites everyone

When a program listens, it chooses which interface to listen on, and this trips up everyone once. `127.0.0.1` (also called `localhost`) means "only this same machine, do not accept connections from the network." Safe by default. `0.0.0.0` means "listen on all interfaces," so other devices, and if your router forwards a port, the whole internet, can reach it. This is a security control disguised as a configuration detail. Bind your AI model and internal services to `127.0.0.1` so they are reachable only on the node itself, and let your reverse proxy (Part 6) be the single thing that listens broadly. Accidentally binding a model to `0.0.0.0` on a machine with a forwarded port is how people expose private services to the whole internet without realising.

You can check what is listening, and on which interface, with one command:

```
sudo ss -tlnp
```

It lists every listening socket, its address, and the program behind it. An entry showing `127.0.0.1:11434` is safe (local only). An entry showing `0.0.0.0:11434` is reachable from your whole LAN, and from the WAN if that port is forwarded. Running this command after you start any service, and confirming it binds where you expect, is a thirty-second habit that prevents the most common self-inflicted exposure in this entire book.

Pointer. Paste the output of `sudo ss -tlnp` to your model and ask: “For each listening socket here, tell me whether it is reachable only from this machine, from my whole local network, or potentially from the internet, and flag anything that looks like it should not be exposed.” This turns a wall of output into a security review you can act on.

3.3 NAT: how one public IP serves a whole house

Your home has many devices but usually one public IP. Your router performs **NAT** (Network Address Translation): it lets all your private devices share that one public address for out-bound traffic, keeping a table of who asked for what so replies find their way back. A crucial side effect: NAT is a one-way valve by default. Your devices can reach out, but the outside world cannot reach in, because the router has no entry for an unsolicited inbound connection and simply drops it. This is why your home node is, by default, invisible from the WAN, a real if accidental layer of protection. It is also why “works at home but not on cellular” happens: at home you are on the LAN, talking directly; on cellular you are on the WAN, knocking on a door NAT has not been told to open. Making your node deliberately reachable means punching one specific hole, port forwarding, which is Part 6.

3.4 IPv4 versus IPv6, briefly and honestly

The old scheme, IPv4 (Internet Protocol version 4), has about four billion addresses, not enough for the planet, which is exactly why NAT exists: to share scarce public IPv4 addresses across many devices. The newer IPv6 (Internet Protocol version 6) has effectively unlimited addresses, enough to give every device its own public address, which means IPv6 often has no NAT and devices can be directly reachable. Three practical consequences. Most home reachability today still runs over IPv4 plus port forwarding, because it is universal, so start there. If your provider gives you IPv6, your node may be reachable from outside without port forwarding, which is convenient and also means the router’s NAT is no longer accidentally protecting you, so your firewall (Part 7) is doing the protecting and must be made explicit. And if your provider uses carrier-grade NAT (CGNAT), where even your public IPv4 is shared, you cannot port-forward at all, and the fix is a different reachability strategy noted in Part 6. The honest summary: IPv4 is the well-trodden path, learn it first; IPv6 is the future and changes who is protecting you, so when you use it, make your firewall do the job NAT used to do by accident.

3.5 Finding your way around your own network

Three small skills make the rest of the book smoother, and each is one command plus a habit of reading the result.

Find your node’s address on the LAN, run on the node:

```
ip addr
```

Look for an address starting 192.168. or 10. on your wired or wireless interface. That is how every other device in your house will reach the node.

Find your public address, the one the WAN sees, run on the node:

```
curl -s https://api.ipify.org; echo
```

If this address matches what your router's admin page reports, you have a normal public IP and port forwarding will work. If it does not match, you may be behind carrier-grade NAT, which is the case to handle in Part 6.

Test whether a port is actually reachable, run from another machine:

```
curl -v http://NODE_IP:PORT/
```

If it connects, the door is open and a service is answering. If it hangs or refuses, either nothing is listening, the service is bound to localhost only, or a firewall is blocking it, and you now know which three rungs to check. This is the abstraction ladder applied to networking: connection refused versus connection hangs versus wrong response each point at a different layer.

3.6 Wires and radio: why CAT6 and WiFi are enough for a home

Your home node should be on a cable, and your roaming devices on WiFi. Here is why, with the limits that matter.

Copper Ethernet (CAT5e, CAT6, CAT6a) carries gigabit Ethernet up to **100 metres** under the standard wiring rules. For ten-gigabit Ethernet, plain CAT6 is limited to about **55 metres** by crosstalk, while CAT6a holds the full 100 metres. A home is far inside these limits: the longest run in a house is a few tens of metres, so a single CAT6 cable from your router to your node gives stable, high, interference-resistant bandwidth with margin to spare. Use solid-core cable for in-wall runs (lower resistance, full distance) and stranded patch cables for the last short hop.

WiFi serves the devices that move. Real-world single-device throughput near the router is roughly 1.1 gigabits per second on WiFi 6 and about 1.8 on WiFi 6E, measured a few metres away, well below the headline “theoretical” figures of around 9.6 (WiFi 6 and 6E) and 46 (WiFi 7) gigabits, which almost no real setup reaches. Range is similar across WiFi 6, 6E, and 7, on the order of 70 metres indoors at the outer fringe, but speed falls fast through walls, and the 6 GHz band in particular can drop off past about 15 metres through residential walls. The lesson is simple: the 2.4 GHz band reaches furthest and penetrates best, the 5 and 6 GHz bands are faster but shorter, and for a large home needing several access points the right move is to run cable between them rather than chain radios.

Why fiber is not needed in this book. Fiber's advantage over copper is distance: it carries signal far past copper's 100-metre wall without repeaters. A home never needs that. The only place it becomes relevant is connecting separate buildings, runs of hundreds of metres or more, which is a multi-site concern. A single-building school (Volume 2) is still copper and WiFi. A university spread across many buildings (a later volume) is where fiber between sites earns its place. For Volume 1, the rule is: copper to the node, WiFi to the roamers, no fiber.

There is a principle here that outlives the specific numbers: match the medium to the distance and the need. Inside one building, copper is cheaper, simpler, and more than fast enough, so use it for the things that sit still and want stable bandwidth (your node), and use radio for the things that move (your phone, your laptop). Reaching for fiber at home is like renting a cargo ship to cross a pond. The scale of the problem, not the prestige of the technology, picks the tool. That instinct is exactly what scales across the volumes: the home wants copper and radio, and only when the radius grows past a single building does the next tool become the right one.

3.7 Understanding your router, the one box that matters

Your router is the single most important piece of networking hardware you own, because it is the border between your LAN and the WAN, and almost every reachability decision in this book happens inside it. It does several jobs at once: it hands out private LAN addresses to your devices (DHCP, Dynamic Host Configuration Protocol), it performs NAT so they share one public address, it runs the WiFi, and it decides what inbound traffic, if any, is allowed in (port forwarding and its own firewall). You reach its control panel by opening its LAN address in a browser, usually 192.168.1.1 or 192.168.0.1, and logging in.

A few things are worth doing on day one, regardless of the rest of the book. Change the router's admin password from the default, because the default is public knowledge and a router with a default password is a wide-open front door. Give your node a fixed LAN address (a DHCP reservation), so its address never changes under you and your port forwarding never points at the wrong device. Note your WiFi security is WPA2 or WPA3 with a strong passphrase, because that passphrase is the fence around your whole trusted LAN from 0.2. And find, but do not yet touch, the Port Forwarding section, because that is the one door you will deliberately open in Part 6.

The mental model: the router is the reception desk for your whole house, and everything about who can reach what passes through it. Understanding it is most of understanding home networking, and it demystifies the magic, because once you have seen the DHCP table, the NAT, and the port-forward page, "how does the internet reach my server" stops being mysterious and becomes a setting you control.

3.8 DNS, a little deeper, and a quiet privacy leak

DNS is the phone book that turns a name into a number, and it runs constantly and invisibly: every time any device asks for a name, it asks a DNS server to resolve it. By default, that DNS server is usually your internet provider's, which means your provider sees a list of every name every device in your house looks up, which is a meaningful privacy leak even when the connections themselves are encrypted, because the lookups reveal where you are going before you get there.

Two consequences for a sovereign-minded home. First, you can choose your DNS resolver rather than defaulting to your provider's, and you can run a local resolver on your own node that handles lookups for your whole house, which keeps that list at home and can also block known tracking and ad domains for every device at once. That is a natural early agent-like service for a home node, and it fits the thesis precisely: it moves one more thing you were unknowingly outsourcing back under your own roof. Second, and connecting to Part 5, this is the same instinct as the offline Wikipedia: the more of your everyday lookups (knowledge, names, answers) that resolve locally, the less of your life is narrated to someone else's logs in real time. None of this is required to build the node in Part 6, but it is exactly the kind of thing you will want to bring home once you have felt how good local-first is.

Pointer. To understand your own DNS exposure, ask: "Explain what my DNS resolver can see about my browsing even when my connections use HTTPS, how to find out which DNS server my devices are currently using, and what my options are for running my own local resolver at home so those lookups stay under my control. Keep it concrete for a home network." This is the kind of invisible dependency the whole book is about noticing and reclaiming.

PART 4: THE LOCAL AI

The demanding tenant. The goal of this part: a model running on your node, answering on a local socket, with the pieces in place to give it memory and senses.

4.1 The runner

You do not hand-wire neural networks. You use a runner that downloads, manages, and serves models behind a simple local API (Application Programming Interface). The gentlest on-ramp downloads a model and serves it on a local port in three steps of the form: install the runner, pull a model, run it. The runner then exposes an HTTP API on a local port (a common default is 11434), which is how your apps, your agents, and the retrieval pipeline in Part 5 will talk to it.

Once it is running, you can talk to it two ways. Interactively, in your terminal, you simply type and it answers. Programmatically, over its local HTTP API, which is the part that matters for building, you send it a request exactly like the request-response dance from Part 3:

```
curl http://127.0.0.1:11434/api/generate -d '{
  "model": "your-model-name",
  "prompt": "Explain what a reverse proxy is in two sentences.",
  "stream": false
}'
```

That is your own AI, on your own machine, answering over a local socket, with nothing leaving the house. Every clever thing later in this book is built on top of exactly this call. The agents, the Wikipedia retrieval, the log summaries: all of them are this request, with different prompts and some surrounding code.

Pointer. Once your runner is up, ask your local model to introduce itself to you: “You are running locally on my own machine now. In plain language, explain what just happened technically: where you are loaded in memory, why you answer faster or slower depending on the hardware, and what it means that nothing I type to you leaves this computer.” Hearing your own private model explain its own privacy is the moment the whole book clicks.

4.2 Picking a model to fit your memory budget

Model size is quoted in parameters, in billions. Raw, each parameter wants about two bytes, so an 8-billion model is about 16 GB, too big for most consumer cards. The rescue is **quantization**: compressing weights to roughly four bits each (written Q4) with little quality loss. In practice a four-bit quantization averages a little above four bits per weight once higher-precision layers and small per-block scales are counted, so an 8B model lands around 4.8 GB, a 14B about 9 to 10 GB, a 32B about 18 to 20 GB, and a 70B about 40 GB. Match the model to your fastest memory with headroom, and remember the lens from Part 1: smaller-but-resident beats bigger-but-spilling every time, because a model that overflows your memory collapses from tens of tokens per second to single digits. Start small (a 3B or 8B), confirm it

runs well, then climb only as far as your hardware genuinely supports.

A practical rhythm: pull a small model first and confirm the whole pipeline works end to end before you reach for size. A fast small model that runs is infinitely more useful than a large one that will not load, and you can always climb later once you have proven the plumbing. Quality has also climbed steeply, so a good 8B or 14B model in 2026 does work that needed far larger models a year or two earlier, which means the sweet spot for a home node is often a mid-size model that fits comfortably with room for a long conversation's context.

4.3 Senses: local speech

A model that only reads text is half-blind. Add local speech-to-text so the node can listen, turning recordings into transcripts entirely on-device, with no audio ever leaving the house. Run it as a systemd service bound to 127.0.0.1 on its own port, and your reverse proxy will route to it. This is the same backend a browser-based voice recorder would call, and it keeps the whole listening pipeline private by construction. You will wire exactly this into the voice recorder app you serve in Part 6, so the recording happens in the browser and the transcription happens on your node, and the audio never touches anyone else's computer.

4.4 Memory: embeddings and a vector store

Out of the box, a model forgets everything between conversations. To give it durable, searchable memory of your own material, you use two more local pieces. An **embedding model** turns any text into a vector, a list of numbers that captures its meaning, so that similar meanings land near each other in that number-space. A **vector store** holds those vectors so you can ask "what do I know that is relevant to this" and get the closest matches back. Together they enable retrieval, which is the on-ramp to Part 5, and they are the difference between a generic assistant and one that knows your world.

The intuition worth keeping: an embedding turns meaning into a location. Two sentences that mean similar things end up near each other in a high-dimensional space, even if they share no words. So "how do I make my node reachable from outside" and "steps to expose a home server to the internet" land close together, and a search by meaning finds the second when you ask the first. That is why retrieval feels like the model understands your question rather than just matching keywords: it is searching by meaning, not by spelling.

4.5 What an "agent" actually is

Stripped of hype, an agent is a loop:

```
perceive -> read some input (a message, a sensor, a log line, a file change)
decide   -> ask the model what to do about it
act      -> run a tool, write a file, send a message, call another service
report   -> record what happened, and surface it to you
then back to perceive.
```

That is all. The model supplies the decide step; your code supplies the perceive, act, and report scaffolding and the tools the model is allowed to use. An agent that watches a folder and summarises new files is this loop. An agent that tails your security logs and warns you of probing (Part 7) is this loop. An agent that checks your feeds and writes you a morning digest (Part 8) is this loop. Once you see the loop, the whole "agent" world stops being magic and becomes plumbing you can build, and (per Part 9) test.

Your first agent can be tiny and still teach the whole pattern: a script that watches a folder, and whenever a new text file appears, sends its contents to your local model with the prompt “summarise this in three bullet points” and writes the summary beside it. That is perceive (a new file), decide (ask the model), act (write the summary), report (the summary file is the report). Build that once, wrap it in a systemd service so it runs forever, and you have understood agents more deeply than most of the people who use the word, because you have seen that the intelligence is borrowed from the model and the autonomy is just a loop you wrote.

Pointer. Ask your model to scaffold your first agent: “Write me a small script that watches a folder for new text files and, for each one, calls my local model’s API to summarise it into three bullets, then saves the summary next to the original. Explain the perceive-decide-act-report loop as it appears in the code, and show me how to run it as a systemd service so it never stops.” You are not just getting code. You are getting the loop made visible.

4.6 Talking to a model well: context, system prompt, temperature

Three ideas turn a model from a slot machine into a tool you can aim. First, the **context window** is the amount of text the model can consider at once, your prompt plus the conversation so far plus any retrieved passages. It is finite, and when a conversation or a document exceeds it, the oldest material falls out of view. This is why retrieval (Part 5) matters: rather than stuffing everything into a limited window, you fetch only the few most relevant passages and spend the window on those. It is also why very long conversations start to “forget” the beginning, and why summarising as you go is sometimes the right move. A fuller window costs speed as well as room: the more context the model must attend to, the more work each token takes, so a long conversation not only risks losing its start but also generates more slowly than a short one (the context-fill slowdown from Part 1.2), which is a second reason to keep only what matters in the window.

Second, the **system prompt** is a standing instruction that shapes every answer in a session: who the model should be, what it should assume, what format you want. Setting a good system prompt once (“you are helping me operate a Linux home node; prefer concrete commands; warn me before anything destructive; I am on Arch”) is far more effective than re-explaining yourself every message, and it is the single biggest lever most people never touch.

Third, **temperature** controls randomness: low temperature makes answers focused and repeatable (good for code, facts, and commands), higher temperature makes them more varied and creative (good for brainstorming, worse for anything you need to be exactly right). For operating a node, you usually want it low.

The practical craft underneath all three is unglamorous and powerful: be clear, be specific, give examples of what you want, and tell the model what *not* to do as well as what to do. A short, precise prompt beats a long vague one almost every time. This is a skill, the skill of rung two of the self-reliance ladder, and it transfers whole from cloud models to your own local one.

Pointer. Have your model help you write a reusable system prompt for your node: “I operate a Linux home node and I will be asking you for help with commands, debugging, and configuration often. Write me a system prompt I can set once that makes you maximally useful for this: concrete, cautious about destructive actions, aware that instructions are not guaranteed to work on my exact machine, and inclined to explain which layer a problem is on. Then explain why each part of it helps.” You are tuning your most-used tool.

4.7 Choosing between models, and keeping them current

Models are not all the same, and the right one depends on your hardware and your task. The honest guidance is to optimise on three axes in order: does it fit in your fast memory with headroom (capacity, from Part 1), is it fast enough to be pleasant (the bandwidth lens), and is it good enough at your actual tasks (quality). The first two you can compute; the third you discover by trying. A good habit is to keep two or three models around: a small fast one for quick everyday questions, a mid-size one for harder reasoning, and perhaps a specialised one (for code, say) if you do a lot of one kind of work. Your runner makes switching between them trivial, so you are not locked into one choice.

Keeping current matters because the open-model world moves fast: the best model that fits your hardware today is often noticeably better than the best one from six months ago, at the same size. So periodically pull a newer model and compare it on your real tasks, and retire the old one if the new one wins. Crucially, this is something you do on your terms, locally: there is no forced upgrade, no model deprecated out from under you, no quality silently changed by a provider overnight. You choose when to update and you can always keep the old model if you prefer it, which is exactly the ownership the thesis promised, applied to the AI itself.

4.8 The honest gap: local versus cloud in 2026

A fair book admits what its approach does not yet do. As of 2026, the most capable models still live in the cloud. A frontier cloud model is larger than anything you can hold at home, often noticeably sharper on the hardest problems, and wired to a wider array of tools, and for the most demanding tasks that gap is real, not imagined. Local models trail, and fully closing the distance may take a few more years.

Two things make this far less discouraging than it sounds. First, the gap narrows fast: a good local model in 2026 does work that needed a frontier cloud model only a year or two earlier, and the curve is steep, the same memory-bandwidth-and-better-models curve that put capable AI on a shelf in the first place (Part 1.3). Second, and more practically, almost everything a home node actually does (answering from your own documents, transcribing, summarising, drafting, watching logs, running the small agents of Part 8) does not need the absolute frontier, because those are not frontier-hard tasks. A mid-size local model is already enough for them, today.

So the honest position, and the one this book takes, is not “local has caught up.” It is this: reach for the cloud on the rare occasions you genuinely need the frontier, run everything else locally where privacy and ownership are worth more than the last increment of capability, and watch the boundary between those two move outward every few months. The book bets that “everything else” keeps growing, and so far that bet has only ever paid off. You are building on the side of the line that is winning.

PART 5: RETRIEVAL, AND YOUR OWN OFFLINE WIKIPEDIA

This is the flagship. We teach retrieval, the principle behind giving any model real, grounded knowledge, through the single most motivating example anyone can build: a private, offline, queryable copy of general human knowledge that your local AI can read.

5.1 What retrieval (RAG) is, in one picture

Retrieval-Augmented Generation, RAG, is simpler than the acronym. When you ask the model something, you first retrieve the most relevant passages from a body of text you control, then hand those passages to the model as context and let it answer grounded in them. The model stops guessing from memory and starts answering from your source, and it can point at which passage it used. That is the entire idea: retrieve, then generate.

Why this matters beyond accuracy: retrieval is how you give a fixed model fresh, private, or specialised knowledge without retraining it. The model stays the same; you change what you put in front of it. That separation, a stable reasoning engine plus a knowledge source you own and control, is the architecture of every useful local AI in this book. It also keeps you sovereign: the knowledge lives in your files, on your disk, not baked into someone else's model weights that you cannot inspect or change.

5.2 The flagship: all of Wikipedia, offline, on your shelf

Wikipedia is freely downloadable in full, and this is rung five of the self-reliance ladder from Part 2: the point where your “google” becomes your own. As of 2026, the English Wikipedia held about 7.2 million articles and roughly 5 billion words. You can have all of it, locally, and let your model read it.

The sizes that matter, all approximate and recorded in 2026:

- The standard article dump (current articles, no talk or user pages, no media) is **over 25 GB compressed** and expands to **over 105 GB** of text.
- The actual article text alone is about **63 GB**; the text of all pages (including templates and other namespaces) is about **166 GB**.
- A ready-made offline reader package (which includes images, after optimisation and compression) is about **100 GB**.
- For contrast, the full edit history of every article is about **28 TB**, and the entire media repository is about **924 TB**. You do not want these. You want the article text.

5.3 How it actually works

The pipeline is the same five steps as any RAG, just at encyclopedia scale. Download the article dump. Split each article into passages of a few hundred words (chunking). Run each passage through your local embedding model to get its vector. Store the vectors, with their source text, in a local vector store. Then wire retrieval into your model: a question gets embedded, the store returns the nearest passages, and those passages go to the model as grounding.

None of this touches the WAN once the dump is downloaded.

Walk the steps once concretely so the scale stops being intimidating. The download is a single large file you fetch once and keep. The chunking is a script that reads the dump and writes out passages, which takes a while but runs unattended. The embedding is the longest step, because every passage passes through your embedding model, and on home hardware this is an overnight job for the full encyclopedia, which is fine because you do it once. The storage is just writing those vectors to a local database file. And the query side, the part you use daily, is fast: your question becomes one vector, the store finds its nearest neighbours in milliseconds, and those passages plus your question go to your local model. You built it once, overnight, and now you own an offline, private, queryable Wikipedia forever.

The storage budget is reassuring. The text you embed is on the order of 60 to 105 GB. The vectors and stored passages add roughly another 100 to 200 GB depending on how finely you chunk and how large your embedding dimension is. Call the whole offline-Wikipedia-RAG footprint somewhere around **150 to 300 GB**. That means even the cheapest tier in the next section, with a single 1 TB SSD, holds the entire encyclopedia plus its embeddings with room left over, and a 2 to 4 TB drive gives you space to keep monthly snapshots, add your own articles and edits, and store backups, all locally.

Pointer. When you are ready to build it, have your model plan the pipeline for your exact hardware: “I want to build an offline Wikipedia RAG on my machine, which has [your specs]. Walk me through downloading the article dump, chunking it into passages, embedding them with a local model, storing them in a local vector database, and querying them through my local LLM. Tell me roughly how long the embedding step will take on my hardware and how much disk each stage needs, and warn me about the common mistakes.” Then build it step by step, asking when a step does not behave.

5.4 The budget, in 2026 euros

What does it cost to run a local Wikipedia-RAG plus a capable model at home? Three tiers, framed by the bandwidth lens from Part 1. All prices were recorded in 2026 for EU buyers, VAT (Value Added Tax) included, and they move; the reasoning does not. Speeds are single-user interactive tokens per second on small-to-mid models, which is what a home user actually experiences asking one question at a time.

Tier 1, under 1,000 euros: it works. The anchor is a used RTX 3090, which traded in the EU around 750 to 950 euros through 2026, giving you 24 GB of VRAM at roughly 936 GB/s in a basic host (a modest CPU, 32 to 64 GB of system RAM, an 850-watt power supply, a case, and a 1 to 2 TB SSD). On this you get roughly 95 to 112 tokens per second on an 8B model and can run up to a 32B model at Q4. Wikipedia and its embeddings fit on the SSD with ease. The honest limit: a 70B model does not fit in 24 GB and will crawl. The alternative at this budget is a quiet mini-PC or a base unified-memory machine, which is silent and sips power but generates more slowly. Either way: usable, patient on the biggest models, fast on small ones.

Tier 2, around 2,000 euros: the sweet spot. Two paths. Either a stronger single-GPU build (a used 3090 or a current mid-to-high card, a better CPU, 64 GB of RAM, a 4 TB SSD), or two used 3090s for 48 GB of combined VRAM, which holds a 70B model at Q4 and generates it at roughly 15 to 20 tokens per second, comparable to a much pricier unified-memory machine. Alternatively a base unified-memory desktop (an Apple Mac mini or Studio in the mid configuration) gives silence and 70B capability with patience. This is the tier most people should pick: comfortable interactive speed on 8B to 32B models, the ability to reach 70B if you want it,

and a 4 TB drive that holds Wikipedia, retention snapshots, your own additions, and backups without thought.

Tier 3, around 4,000 euros: faster, bigger, before diminishing returns. Either raw speed or large capacity. For speed, an RTX 5090 (street prices in 2026 sat well above its roughly 2,000-dollar launch price, often 2,600 dollars and up, with partner cards higher) in a strong build gives about 186 tokens per second on an 8B model and runs 32B comfortably, though a 70B still will not fit on its 32 GB. For capacity, a 128 GB unified-memory machine (around 4,000 to 4,500 euros) holds 70B-class models entirely in fast memory and generates them at roughly 10 to 15 tokens per second, in near silence, drawing a fraction of a discrete card's power. Pick speed if you mostly run models up to 32B; pick capacity if you want the largest models resident.

Why 10,000 euros buys a home almost nothing more. Past about 4,000 euros, a single-user home node hits diminishing returns, and it is worth being precise about why. At Tier 3 you already run 70B-class models and get interactive speeds. The next step up (more or faster GPUs, more unified memory) mainly buys three things: even larger models above 70B that a home user rarely needs; higher batch throughput for serving many concurrent users, which a household of one to four people does not have; and training or fine-tuning, which is out of scope. A 10,000-euro home box gives marginally bigger models or faster concurrent serving that a single person literally cannot perceive, because the model is already faster than your reading and typing speed can consume. You are saturated. The bottleneck has quietly moved from “can it run” to “do I need it,” and at home the answer is no.

This is exactly where the scale tips toward the next volume. A school is not one user, it is hundreds of students and teachers asking at once, plus central storage for everyone's files and results, plus the need for redundancy so the whole institution does not stop when one part fails. There the constraint flips from single-stream latency to aggregate throughput, capacity, and reliability, and 10,000 euros becomes the floor for bare Information Technology (IT) services rather than the ceiling. That institution is Volume 2. For one human and one home, Tier 2 is plenty and Tier 3 is luxury.

5.5 Beyond Wikipedia

The same pipeline ingests anything. Point it at your own notes, your books, your code, your saved articles, and your model answers grounded in your world instead of a generic one. Wikipedia was the proof that the method scales to millions of documents on a shelf. Everything after it is the same five steps with a different corpus.

This is also where the privacy story becomes vivid. A cloud assistant that “knows your documents” has your documents. A local RAG that knows your documents keeps them on your disk, embeds them on your machine, and answers from them with your own model. Same capability, opposite custody. You get an assistant that knows your life without anyone else knowing your life, which is the entire promise of this book reduced to one feature.

5.6 Chunking and embedding choices, and why retrieval sometimes misses

Retrieval quality comes mostly from two choices you make when building the index, so it is worth understanding them rather than treating retrieval as a black box. The first is **chunking**: how you split documents into passages. Chunks that are too large dilute meaning (a passage about ten things embeds as a blur and matches nothing well), while chunks that are too small lose context (a single sentence may be meaningless without its surroundings). A few hundred words per chunk, often with a little overlap so ideas that straddle a boundary are not cut in half, is a sensible default. The second is the **embedding model**: different embed-

ding models capture meaning with different fidelity, and a better embedder makes “search by meaning” noticeably sharper. Both are tunable, and both reward a little experimentation against your own questions.

It also helps to know why retrieval sometimes misses, because then you can fix it instead of distrusting the whole approach. If a question and the relevant passage use very different language, their vectors may not land close enough, and the right passage is not retrieved, so the model answers from its own memory instead, which can be wrong. The cures are practical: retrieve more candidates and let the model sort them, improve the chunking so passages are coherent, or use a better embedding model. The mental model to keep is that retrieval is search by meaning, and like any search it can fail to find what is there, so a good RAG system retrieves generously and lets the model judge, rather than betting everything on the single nearest match.

Pointer. When your retrieval gives a wrong or empty answer, debug it the same way you debug anything: “My local RAG answered [wrong thing] for the question [question]. Help me figure out whether the retrieval failed (the right passage was not fetched) or the generation failed (the right passage was fetched but the model ignored it). Tell me how to inspect which passages were retrieved, and based on that, whether I should change my chunking, retrieve more candidates, or use a different embedding model.” Retrieval failures are debuggable, not mysterious.

5.7 Keeping your offline Wikipedia fresh

Wikipedia changes every day, so a copy you downloaded once slowly ages. This is not a problem so much as a choice about cadence, and it connects directly to retention and backups. You decide how fresh you need general knowledge to be: for most home use, refreshing every few months is plenty, because the core of the encyclopedia (history, science, concepts) barely moves, and only current events drift quickly. Refreshing is the same pipeline as building, pointed at a newer dump: download the new article dump, re-chunk and re-embed it (the overnight job again), and swap it in.

The genuinely nice part, and a quiet argument for the larger drives in the budget tiers, is that you can keep snapshots. Because the whole corpus is only a couple hundred gigabytes and storage is cheap, a 2 to 4 TB drive lets you keep several dated copies of Wikipedia rather than just the latest, which means you hold not only what the encyclopedia says now but what it said at past moments, a small private archive of how knowledge changed. That is something even the live website cannot easily give you, and it falls out for free from owning your own copy. Add your own corrections and additions on top (Part 5.5), keep them in your retention, and you have an encyclopedia that is both general and yours.

PART 6: MAKING IT REACHABLE, SAFELY

This part crosses the border from LAN to WAN. Everything tightens here, because you are now inviting the public internet to knock on your door. Do it deliberately, in this order. And then, at the end, comes the twist that the whole book has been pointing at: once you can reach your node from anywhere, you will see clearly that the best part is that it does not depend on that reach.

6.1 The reverse proxy: a doorman that stores nothing

Do not expose your app, model, or services directly. Put one program in front of all of them, a reverse proxy, whose entire job is to listen on the public ports (80 and 443), terminate HTTPS, and route each request to the right internal service on `localhost`. It stores no user data; it holds only configuration. It is the single front door, and the only thing the outside world ever talks to. The classic choice is `nginx`; the modern easy choice is `Caddy`, which fetches and renews HTTPS certificates automatically. The design that matters: your model and speech service listen only on `127.0.0.1`, so the world cannot reach them directly, only through the proxy, on the paths and terms you allow.

The mental picture is a building with one staffed reception desk. Every visitor enters through reception, states which office they want, and reception walks them there. The offices themselves have no doors to the street. That is your node: many services inside, all on `localhost`, and exactly one reception (the proxy) facing the world, deciding who reaches which office. Get this shape right and the rest of security gets dramatically simpler, because there is only one door to watch.

6.2 Walkthrough A: your first page, “Hello, World,” served from your own node

This is the moment the abstract becomes real. We will serve a single web page from your node, first to your own LAN, then, once it works, to the whole WAN over HTTPS. Follow the shape; ask your model to adapt each step to your exact distribution and proxy.

Step 1: make the page. On the node, create the smallest possible website, one file:

```
mkdir -p /home/you/sites/hello
echo '<!doctype html><meta charset="utf-8"><title>Hello</title><h1>Hello, World. ↵
    ↵ This is my node.</h1>' > /home/you/sites/hello/index.html
```

That is a complete web page. It does nothing clever. It exists to prove the pipeline.

Step 2: serve it on the LAN, over plain HTTP. Point your reverse proxy at that folder, listening on port 80, serving the file. With `nginx`, this is a short server block that says “for requests to this node, serve files from `/home/you/sites/hello`.” With `Caddy`, it is a two-line config naming the folder. Reload the proxy.

Step 3: see it from your own couch. On the node, find its LAN address with `ip addr` (something like `192.168.1.50`). Now, from your phone or laptop on the same WiFi, open `http ↵
 ↵ ://192.168.1.50/` in a browser. Your page appears. Stop and notice what just happened:

a device in your house asked your node for a page, and your node answered, entirely over your LAN, never touching the internet. You are already serving. You already have a working local web server, reachable by every device in your home, depending on no outside party whatsoever.

Pointer. If the page does not appear, debug by the ladder: “My browser cannot load `http://192.168.1.50/`. Walk me down the layers: is the proxy running, is it listening on port 80 on all interfaces or only localhost, is the firewall allowing port 80 on the LAN, is the file path correct and readable, and what command checks each one? I am on [your distribution] using [nginx or Caddy].” The fix is almost always one rung, and naming the rungs finds it fast.

Step 4: cross to the WAN, on purpose. Now make that same page reachable from anywhere, which means everything from Part 0 about the WAN being hostile now applies, so HTTPS becomes mandatory. Three things happen together, covered in 6.3 to 6.5: you get a domain name and keep it pointed at your changing home IP (dynamic DNS), you forward ports 80 and 443 on your router to your node, and you turn on HTTPS so the now-public page is encrypted. When those are done, you open `https://yourname.example/` from cellular, far from home, and your “Hello, World” appears, served from the box on your shelf, over an encrypted connection, with no rented computer doing the work in between, only the dumb pipe that carries it, blind to what it carries. That is the entire critical path of this book, proven with the simplest possible payload. Everything after is just a more interesting page.

6.3 A name and a moving target: domain plus dynamic DNS

You want a name, not a number, and there is a wrinkle: most home connections have a dynamic public IP that changes periodically, which would leave DNS pointing at the wrong place. The fix is **Dynamic DNS**: a small client on your node (or a feature in your router) watches your public IP and updates your domain’s DNS record whenever it changes, so the name always finds you. Set it up once and forget it; the name follows your home address around automatically.

6.4 The hole in the valve: port forwarding

Recall NAT (Part 3): inbound connections are dropped by default. To let the WAN reach your proxy, you port-forward ports 80 and 443 on your router to your node’s private LAN IP. This is a configuration change only you can make, by logging into your router’s admin page (often at `192.168.1.1` or `192.168.0.1` in a browser) and finding the Port Forwarding section. Give the node a static LAN IP, or a DHCP reservation in the router, so the forward never points at the wrong device. Now a request from anywhere on Earth hits your public IP, NAT forwards 443 to your node, and your proxy answers over HTTPS. You have crossed the border on purpose, through one guarded door.

Two honest footnotes. On IPv6 there is often no NAT and thus no port-forward, so your firewall (Part 7) decides what is allowed in. And if your provider uses carrier-grade NAT (which you detected in Part 3.5 when your public IP did not match your router’s), port forwarding will not work at all, and the fallback is an outbound tunnel: a relay that holds a public endpoint and forwards traffic down a connection your node opens from the inside. That is a small dependency for reachability only, with your data and your compute still entirely at home, which keeps the core promise intact even when your provider will not give you a real door.

6.5 HTTPS, and the key in the URL

You are on the WAN now, so HTTPS is mandatory. The good news is that it is free and automatic. With nginx, a certificate tool obtains and auto-renews a free certificate from a public certificate authority and edits your config to add the certificate and an HTTP-to-HTTPS redirect. Caddy does the same with no command at all, just by naming the domain in its config. Either way, from here on, traffic crossing the hostile WAN to your node is encrypted, and a stranger on the wire sees only ciphertext.

It is worth sitting with how strong that is, because it is the quiet foundation of the whole “private even on the WAN” claim. The pipe you rent is untrusted by assumption: your internet provider, the mobile network, and everyone whose equipment your packets cross can see that traffic flows and can even store every bit of it forever. Encryption means none of that matters. They can keep all the ciphertext they like and they will never read it, because recovering your data would require finding the key, and the key is one specific value out of a space so vast that trying them all is not merely slow but effectively impossible, more combinations than there are atoms available to count them with. That is the magic the S in HTTPS buys: not hiding that you are talking, but making what you say physically infeasible to recover without the key. So you can rent the dumbest, most hostile pipe on Earth and still cross it in total privacy. Renting transport is not renting trust.

For a personal service you may not want a public login at all. A powerful, legitimate pattern is to serve your app only at a long, high-entropy URL (Uniform Resource Locator) that acts as a capability token: the URL is the key. This is not “security through obscurity” in the bad sense (which means hiding the mechanism). It is the same principle a private key relies on: enough random bits that brute-forcing the address is computationally hopeless. It is used everywhere in production, from “anyone with the link” documents to signed download URLs. Do it right, and be precise about the trade:

- **The entropy must be real.** Generate the token from a cryptographic random source, on the order of 128 bits or more. “Long” is not “random,” and re-encoding adds no entropy. A timestamp, or anything from a non-cryptographic random function, is guessable no matter how long the string looks.
- **It is a bearer token.** Whoever holds the link gets in. It authorises possession, not identity, so there is no per-user audit or revocation without rotating the key.
- **It leaks where secrets leak.** Unlike a private key, which never crosses the wire (you prove possession by answering a challenge), a URL token is sent on every request and comes to rest in browser history, bookmarks, and, pointedly, your server logs. The monitoring you use to catch attackers (Part 7) is the exact place your key piles up in plaintext, so treat those logs as sensitive as the key, and rotate.
- **Rotate or expire it.** A token that changes every few hours turns a long-lived secret into a short-lived one and shrinks the leak window toward nothing.
- **Right-size it to the stakes.** This model shines precisely when there is nothing behind the door worth stealing, which is the drawer model of Part 0: the server stores no personal data, so a leaked key buys an attacker access to a stateless service, not anyone’s data, and the cost is some compute, not a breach. The moment there is anything sensitive behind the door, graduate to real authentication: accounts and signed tokens. Never confuse “good enough for my own notes” with “good enough to be trusted with someone else’s information.”

6.6 Walkthrough B: page two is an entire app, the voice recorder

Now the payoff. Your second page is not a paragraph of text. It is a real, working application: a browser-based voice recorder that records audio, plays it back while you are still recording, and transcribes it using the local speech service from Part 4, with the audio and transcript living only in your browser and the node storing nothing. It is the drawer model of Part 0 as a finished product, and you add it in one move.

The shape of the move, which you adapt with your model to your exact setup:

Step 1: put the app on the node. The voice recorder is a self-contained set of static files: an HTML (HyperText Markup Language) page, some JavaScript that handles recording in the browser, and a small amount of styling. Copy that folder onto the node, alongside your hello page:

```
/home/you/sites/voice/      <- the app's files (index.html, js, css)
```

Because it is just static files served to the browser, the heavy lifting (capturing the microphone, buffering audio, playing the live preview) happens on the visitor's own device, in their browser. The node only serves the files and, when asked, transcribes.

Step 2: give it a route through the proxy. Add a second location to your reverse proxy so that one path serves the app and another path forwards transcription requests to your local speech service:

- /voice/ serves the app's static files from /home/you/sites/voice/.
- /transcribe/ forwards to the speech service on 127.0.0.1 (the one you ran as a systemd service in Part 4).

Reload the proxy. That is the entire integration. The app, running in the browser, records audio locally, and when you ask for a transcript it sends the audio to /transcribe/, which the proxy hands to your local speech model, which returns text, all without the audio ever leaving your node or the visitor's device for anyone else's server.

Step 3: use it from anywhere, privately. Open <https://yourname.example/voice/> from your phone, far from home. You record a voice note. It plays back as you speak. You tap transcribe, and a moment later the text appears, produced by a model running on the box on your shelf. No cloud transcription service saw your voice. No account. No upload to anyone. You built a private voice-notes app with live transcription, reachable securely from anywhere, depending on nobody, and it was page two.

Notice the architecture you just demonstrated, because it is the template for everything you will build next. The browser does the device-side work and holds the data (the drawer). The node serves the static app and runs the private AI service (the stateless reception plus the local model). The proxy routes between them and provides the encrypted front door. And the WAN is used only as a path to reach your own node, not as a place where any of your data or computation actually lives. That is a complete local-first application, and you now have the pattern to build a dozen more.

Pointer. To wire the app's transcription to your local service, describe both halves to your model: "I am serving a static browser voice-recorder app at /voice/ through my reverse proxy. The app records audio in the browser and needs to POST it to /transcribe/, which should forward to my local speech-to-text service on 127.0.0.1:PORT. Show me the proxy configuration for both

routes, explain how to confirm the audio never leaves my node except to my own service, and help me test the transcription end to end.” You are assembling real functionality out of pieces you already own.

6.7 Walkthrough C: a third page that serves data, a tiny API

Your first page was static, your second was a whole app, and your third shows the last shape you need: a service that returns *data* rather than a page, which is exactly how your agents (Part 8) will talk to each other and how apps talk to your model. An API endpoint is just a URL that, when requested, returns structured data (usually JSON, JavaScript Object Notation) instead of a web page. You already met one: your model’s runner exposes an API on 127.0.0.1:11434, and your transcription route forwards to one. Now you make your own.

The shape, adapted with your model to your language of choice: a tiny program that listens on a local port and answers one route with JSON.

```
# pseudo-shape of a minimal JSON endpoint, bound to localhost
GET /api/health -> { "status": "ok", "uptime_seconds": 12345 }
```

Run it as a systemd service bound to 127.0.0.1 (Part 2.6), then add a proxy route so /api/ forwards to it (Part 6.1). Now https://yourname.example/api/health returns a small JSON object from your node, over HTTPS, from anywhere. That is the entire pattern behind every “smart” thing your node will do: a service listening locally, returning data, reached through the one front door. A health endpoint is the natural first one because Part 8’s monitoring will want exactly this kind of signal, and you have now built the template for all of them.

Pointer. Ask your model to scaffold your first API and wire it in: “Write me the smallest possible web service in [language] that listens on 127.0.0.1:PORT and answers GET /api/health with a JSON object containing a status and the process uptime. Then give me the systemd unit to run it and the reverse-proxy route to expose it at /api/, and the curl command to test it end to end.” You now serve data, not just pages, which is the whole basis of the ecosystem to come.

6.8 Troubleshooting reachability: works on the LAN, not on the WAN

This is the single most common snag when crossing the border, and it is a perfect exercise in the abstraction ladder, so walk it deliberately. Your node serves fine to devices at home but is unreachable from cellular or from a friend’s house. The page on the LAN proves the node, the proxy, and the service are all healthy, so the problem is somewhere in the path from the WAN to your node, and there are only a few rungs to check.

Walk them in order. Is the proxy actually listening on the public interface and not just localhost (sudo ss -tlnp, looking for 0.0.0.0:443)? Is the firewall allowing 443 (sudo ufw status)? Is the port forwarded on the router to the node’s current LAN address (router admin page, and confirm the node’s address has not changed, which is why you reserved it in 3.7)? Does your public IP match what your router reports, or are you behind carrier-grade NAT (the test from 3.5), in which case no port-forward will ever work and you need the outbound tunnel from 6.4? Is your domain’s DNS actually pointing at your current public IP, or did your home IP change and dynamic DNS not update (6.3)? Each of these is one check, and the failure is almost always exactly one of them.

The reason this is worth its own section is that it teaches the deepest lesson in the book one

more time: when something works in one place and not another, the difference between the two places tells you where to look. LAN works and WAN does not, so the bug is on the path that only the WAN uses, which is a short list. You do not guess. You walk the list. And when you find it, you will understand your own border better than most people ever understand theirs.

Pointer. Hand the symptom to your model as a guided checklist: “My home node serves correctly to devices on my LAN but is unreachable from the internet. Give me an ordered checklist to find the break: proxy binding, firewall, router port-forward, carrier-grade NAT, dynamic DNS, and HTTPS. For each, tell me the exact command or page to check and what a healthy result looks like. I will report what I find at each step.” This is the LAN-versus-WAN distinction from Part 0 turned into a repair procedure.

6.9 The payoff: what WAN-optional actually buys you

Here is the twist the whole volume has been pointing toward, and it is worth saying plainly now that you have a working, reachable node. You just spent a whole part making your node reachable from the WAN. The most important thing about that reachability is that your node does not depend on it.

First, dispel the obvious misreading. The argument is not that the internet is fragile. The internet is one of the most robust systems humanity has built, and it rarely fails you. The point is subtler and stronger: **not needing** the WAN, while still being able to use it when you want, is a distinct and underrated form of power. Optionality beats dependence even when the thing you would depend on is reliable. Here is what that optionality concretely buys you, all of which your “Hello, World” and your voice recorder already demonstrate:

- **Privacy by construction.** When the core path does not leave your house, there is no third party to trust, no terms of service to change under you, no log on someone else’s server. Your voice note was transcribed by your own machine. That is not a privacy policy. It is a privacy property, true by architecture rather than by promise.
- **Speed.** A request that stays on your LAN has no round trip across the country and back. Talking to your own node is answered at the speed of your house, not the speed of a distant data center under load. Local is simply faster, every time.
- **Resilience without fear.** Your provider can have an outage, a cloud region can go down, a service you relied on can be discontinued, and your node keeps doing its job, because its job was always local. If your node is a laptop or on a small battery, even a mains flicker does not interrupt it. You are not afraid of these events; you are merely indifferent to them, which is a much better place to stand.
- **No rent, no rate limits, no depreciation.** Nothing in the core path bills you monthly, throttles you after N requests, or announces it is shutting down in ninety days. The thing you built is yours, and it keeps working on its own terms. A cloud service is a tenancy. A home node is ownership.
- **Ownership of the stack you run.** Because the layers you can own are open and sit on your own hardware, no vendor can quietly change the deal, raise the price, mine your data, or remove a feature you depend on. The closed firmware floor beneath every machine, named plainly in Part 0 and the appendix, is the one part you do not control, but it cannot reach up and rewrite the software you run. Everything above it is yours to keep, change, or replace.

And the WAN is still right there when you want it. You reach your node from anywhere. You pull in the public web on your terms, in your formats, at your frequency. You order a

replacement disk from a hardware store when one wears out. The WAN did not become forbidden; it became **opt-in**. You moved your center of gravity home, and now you visit the wider internet as a choice rather than living inside it as a dependency. That shift, from leashed to free-to-roam, is the entire benefit, and you have now felt it twice: once with a page that says hello, and once with a private voice app that listens. Every later thing you build deepens it.

PART 7: CYBERSECURITY FOUNDATIONS

Security is not a feature you bolt on. It is a posture you hold from the first command. The governing principle is one line: expose the minimum, and layer your defenses so no single failure is fatal. Here is the home-node application.

7.1 The firewall: shut every door you do not use

Your node should refuse all inbound connections except the few you have chosen. The gentlest front end is `ufw` (uncomplicated firewall), one `pacman -S ufw` away on Arch:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

Three doors open, everything else shut. On IPv6 this matters more, because there is no NAT backstop, and `ufw` covers both. Pair it with the `127.0.0.1` binding habit from Part 3: even if a port slipped open, a service bound to `localhost` is still unreachable from outside. The firewall and the `localhost` bind are two independent walls, and defense in depth means a single mistake in either one is not enough to expose anything.

Pointer. Before enabling a firewall on a machine you reach over SSH, ask: “I am about to enable `ufw` on a remote server I only access over SSH. Walk me through doing this without locking myself out, confirm the exact rule that keeps SSH open, and tell me what to do if I do get locked out. I am on [your distribution].” Locking yourself out of your own node by firewalling SSH before allowing it is a classic first-timer mistake, and one question prevents it.

7.2 Reading your logs, where the truth lives

A node you do not watch is a node you do not understand. Linux records what happens in two main places, the `systemd journal` (`journalctl`) and text files under `/var/log`. The essential moves: follow the proxy’s log live (`journalctl -u nginx -f`), watch authentication attempts (`sudo tail -f /var/log/auth.log`), watch every web request that hit the door (`sudo tail -f /var/log/nginx/access.log`), and check the basics (`df -h` for disk, `free -h` for memory, `systemctl --failed` for anything that died). What to look for: repeated failed logins from unfamiliar addresses means someone is brute-forcing SSH (with key-only auth they cannot succeed, but you will see them try); a flood of requests for URLs that do not exist means automated scanners fishing for known holes, the normal background noise of the WAN. The deepest debugging skill is exactly this: when something is wrong, go read the log of the layer you suspect, and let it tell you the truth.

This is also the first place your local AI earns its keep operationally, which Part 8 builds on:

instead of scrolling thousands of log lines yourself, you hand them to your own model and ask what deserves attention. The logs never leave the node, the summary is private, and you get a short list of decisions instead of a wall of noise.

7.3 Banning the probers automatically

You do not watch logs by hand forever. You build the agent loop from Part 4 to do it. The standard tool, `fail2ban`, watches your logs and, when an address crosses a threshold of bad behaviour, adds a firewall rule to ban it, with escalating durations for repeat offenders. Two honest notes consistent with this book: with key-only SSH and a stateless service, banning mostly sheds load and keeps logs clean rather than saving you from a breach (your entropy and your empty room already do that); and avoid permanent bans as a default, because addresses get reassigned and you will eventually ban someone innocent, so escalating-but-finite is the humane and practical setting.

7.4 Threat-modelling a home node

Spend five minutes being honest about what you are actually defending against, because undirected paranoia wastes effort and misdirected confidence gets you hurt. Realistically you face automated internet-wide scanners probing for known vulnerabilities and weak passwords (constant, impersonal, defeated by patching, key auth, the firewall, and banning), and availability attacks (below). The drawer model removes mass data theft, because you store no central trove, so a compromise of a stateless relay reaches no personal data. What remains and cannot be fully designed away is **denial of service**: anyone reachable can be flooded with traffic until they cannot answer, and a request you must inspect and drop has already cost you the inspection. No entropy or firewall fixes that; the real mitigations are upstream (a provider's scrubbing, a relay in front, rate limits) or simply leaving (7.6). And client-side custody remains: the data now lives on user devices, inheriting their safety and their fragility. Defend the things that match real threats, accept the residue honestly, and do not burn effort fortifying a room you have already emptied.

The reassuring upshot of the drawer model, made concrete by what you have built: your voice app stores nothing on the node, so even a full compromise of that node exposes no voice notes and no transcripts, because they were never there. You did not have to perfectly defend that data. You arranged for it to not be present. That is the strongest security posture there is, and it came from architecture, not from effort.

7.5 Backups: the bill the cloud used to pay silently

Local-first gives you sovereignty and hands you the responsibility the cloud used to absorb invisibly. There is no central copy keeping you safe anymore. So back up, deliberately, automatically, and off the node: a second drive at minimum, and ideally a copy that lives elsewhere physically, because a fire or theft takes the node and any backup sitting beside it. The discipline worth memorising is **3-2-1**: at least three copies, on two different media, with one kept off-site. Back up your data, your service configurations, and, critically, your SSH and TLS (Transport Layer Security) keys, because losing those locks you out. The privacy win of "the browser or OS (operating system) can wipe it whenever the user wants" is the same coin as "nothing restores it if it is gone." Own both faces.

Pointer. Have your model design a backup that matches the 3-2-1 rule for your setup: "Design me a 3-2-1 backup plan for my home node. I want it automatic, I want it to include my

service configs and my SSH and TLS keys as well as my data, and I want one copy off the machine. Suggest concrete approaches for the off-site copy that do not put my data under a cloud provider's control, and write me the systemd timer that runs it on a schedule." Backups you have to remember to run are backups you will not have when you need them, so automate from the start.

7.6 The kill-switch, and the EM caveat

A genuine advantage of a local-first node: you can sever the outside world and keep running. This is the abstraction ladder used in reverse as a defense. Cut a lower rung (the WAN link) and everything above it keeps working, oblivious, because the upper layers were only ever talking to the layer directly beneath them. Pull the cable to the internet and a truly local-first service does not notice; it was always operating on-device and merely syncing when a path existed. That makes the network optional, which is the strongest possible answer to a remote attack: stop being reachable. It is the kill-switch face of the WAN-optional power from Part 6.9. If something on the WAN is going wrong, you can simply step off the WAN, and your node keeps serving your house.

But be precise, because this is where confidence gets people hurt. Unplugging the cable does not air-gap anything. It removes the one tube you can see. WiFi, Bluetooth, cellular, GPS, and NFC are all still radiating in and out through antennas you usually cannot unplug. The electromagnetic spectrum is full of doors. A true air gap is an EM-quiet gap, no radios, ideally a shielded boundary, which is exactly why genuinely high-security facilities are shielded rooms, not merely unplugged ones. So treat isolation as a hierarchy: unplug the WAN cable, then disable the radios, then shield the box, and pick a rung based on what the room is actually worth defending. In the drawer model, that is often very little, which is the point: you did not harden the room, you emptied it, and an empty room barely needs a door.

7.7 Secrets: where keys and tokens live, and how not to leak them

Your node will accumulate secrets: SSH private keys, TLS certificate keys, the capability tokens from 6.5, perhaps an API credential for the one permitted outside dependency (ordering a part). A few habits keep these from becoming your weak point. Keep secrets out of your code and out of anything you might share: a key pasted into a script is a key that leaks the moment you zip that project for your model to review (Part 9), so put secrets in separate files or environment variables that your code reads at runtime, and never commit them anywhere. Lock their file permissions so only your user can read them (a private key should be readable by you and no one else). Remember from 6.5 that tokens come to rest in logs, so treat your logs as sensitive and rotate tokens periodically. And back up your keys, encrypted, as part of the 3-2-1 plan, because a key is both the thing you must never leak and the thing whose loss locks you out, which is the same double-edged sovereignty as the SSH key in Part 2.

The principle is least exposure: a secret should exist in as few places as possible, be readable by as few things as possible, and live as short a time as possible if it can be rotated. You will not get this perfect, and per the book's own stance you do not need to, but moving in this direction (secrets out of code, permissions tight, rotation on the sensitive ones) removes the most common ways people hand attackers the keys to a door they otherwise could not open.

7.8 Physical security and the human layer

Two threat surfaces get forgotten because they are not on the network. The first is physical: your node is a real box in a real place, and someone with physical access to it can often bypass much of your careful network security. For a home this is usually a low concern (your home's

own door is the control), but it is worth a thought, especially for the drawer-model insight in reverse: the data that lives on your devices is as safe as those devices are physically, so a stolen laptop is a data exposure unless its disk is encrypted. Full-disk encryption is the answer, and it is the physical-world version of HTTPS: it makes the data meaningless to whoever holds the hardware without the key.

The second is the human layer, which is, honestly, where most real compromises actually happen. No firewall stops you from being talked into running a malicious command, entering a credential into a convincing fake, or pasting a secret where it does not belong. The defenses are habits, not software: be suspicious of urgency and of instructions that arrive out of nowhere, verify before you run things that touch secrets or money (the single human click in Part 8.4 is an instance of this), and remember that the same skepticism you would apply to a stranger's advice applies to a slick message or a too-good link. The whole book has been training one half of this (understand your system well enough that you are hard to fool), and naming the other half (slow down when something asks for access or money) completes it. A sovereign operator is hardest to compromise not because their walls are highest but because they understand what they are running and pause before they are rushed.

7.9 What the open internet does to an exposed port

Before you forward a port (Part 6.4) and cross onto the WAN, understand plainly what you are stepping into, because the open internet behaves in a way that surprises newcomers and changes how you should configure everything.

The first fact: the moment a port on your node is reachable from the WAN, it will be found, fast, by machines, not people. The entire IPv4 address space is small enough that automated scanners sweep all of it continuously, around the clock, cataloguing every reachable address and every open port on it. Being obscure protects you from nothing, because nobody is hunting for you specifically; everybody is scanning everyone, all the time. Assume that any port you open is discovered within minutes to hours, and that discovery itself is not an attack, it is just the weather.

The second fact: every open port is then probed by the protocols conventionally associated with its number. Port numbers are a shared convention about what lives where: a connection to port 22 expects SSH, 3389 expects remote desktop, 3306 expects a database, 25 expects mail, and so on. So scanners assume the convention and fire the matching playbook at whatever they find. An open 22 draws a relentless stream of automated SSH login attempts. An open 3389 draws remote-desktop attacks. An open database port draws attempts to connect with default credentials. None of it is targeted; all of it is automatic; and all of it arrives simply because the port number advertised which protocol to try.

Two consequences shape your setup. The first is why the defenses in this part are built the way they are: the attempts are constant, automated, and impersonal, so you defeat them by not being guessable and not being open, key-only SSH so the login attempts cannot succeed (Part 2.5), and a default-deny firewall so unused ports are not reachable to probe in the first place (Part 7.1). You do not defeat scanning by hoping to go unnoticed, because you will be noticed; you defeat it by making being noticed harmless.

The second consequence is a practical lever: you can cut the sheer volume of automated noise dramatically by moving the services only you connect to off their default ports. Run SSH on some nonstandard port instead of 22 and the flood of automated 22-attempts simply misses you, because the scanners mostly try the conventional port for each protocol and move on. Be honest about what this buys, in the book's usual spirit: it is noise reduction, not real security. A determined attacker scans every port and finds your service wherever it lives, so a nonstan-

dard port is not a wall, it is a quieter street. But quieter logs make real problems easier to see, and fewer automated attempts is worth a little, so changing defaults for your private services is a reasonable habit, as long as you never mistake it for the actual protection, which is the keys and the firewall.

The one exception, and it is an important one: do not move the web off its defaults. Ports 80 (HTTP) and 443 (HTTPS) must stay where they are, because browsers assume them. When a person types your address, their browser tries 443 automatically; move it and you break the one thing that has to just work for a human visitor. So the rule that falls out is clean: change the default ports for the services only you reach (like SSH), and keep the default ports for the services the world reaches through a browser. Hide your private doors in quieter places; leave your public door exactly where everyone knows to knock.

Pointer. Before opening anything to the WAN, have your model show you the reality: “I am about to forward a port and expose a service to the internet. Explain what automated scanning will do to it within the first day, which ports attract which automated attacks, and how to move my private services (like SSH) off their default ports while keeping web on 80 and 443. Then tell me honestly which of these steps is real security and which is only noise reduction.” Knowing the difference is the whole point.

7.10 Versioning: the backup that remembers every step

Backups (Part 7.5) keep you safe from loss. Versioning is the same instinct levelled up: instead of keeping a copy of how things are now, you keep the entire history of how they changed, so you can move to any past state, see exactly what changed between any two points, and undo one specific change without losing everything since. A backup answers “can I get my data back if the disk dies.” Versioning also answers “what did I change last Tuesday, and how do I revert just that,” which is a different and equally important kind of safety.

The standard tool is a version-control system (git is the near-universal choice), and the unit of safety it gives you is the ability to record a snapshot at every meaningful step, with a short note on what changed and why, and then to walk that history forwards and backwards at will. Put your configurations, your service files, your scripts, and above all your test suites and code under version control, and three things become true at once. You can experiment fearlessly, because any change is reversible to the exact line. You can understand your own past decisions, because the history records them. And you make the method of Part 9 real, because “own the tests” only works if the tests have a trustworthy history you can roll back, exactly as the code does.

Versioning also gives you a precise vocabulary for time, which matters the moment more than one copy of a thing exists. A useful discipline, and a real part of the skill: track every internal iteration with a simple counter so you can always say exactly which build you are looking at, but only bump a public version number when you actually share or publish, because a version number is a promise to other people about what changed since the copy they hold. The two answer different questions. The iteration counter answers “is what I am seeing the latest, and when did it last change,” which you ask yourself a hundred times while building. The version number answers “what changed since the release I already have,” which is what everyone else asks you. Keep both, and you can always trace a behaviour you are seeing back to the exact build that produced it, which is half of all debugging once a thing has a history.

This is also where backup and method meet. Versioning gives you per-change recoverability (the method’s safety net), while the 3-2-1 backups of Part 7.5 give you per-disaster recoverability (the loss safety net), and you want both: versioning so a bad edit is a one-command

undo, backups so a dead drive or a burned house is survivable. Keep your version history itself inside your backups, and you hold the entire story of your stack, recoverable from any failure, at any point in its past. That is about as un-fragile as a thing you own can be, and it connects straight back to the recovery mindset of Part 2.8: a node you can always return to a known-good state is a node you can change without fear.

Versioning also reaches naturally off the machine, which is where push and pull come in. A version-control system can sync its history to a remote, a hosted service like GitHub or a small git server on another box you own, by pushing your snapshots up and pulling changes back down. That remote is two useful things at once: an off-site copy of your whole history, which is exactly the off-site leg of the 3-2-1 rule from Part 7.5, and the mechanism by which more than one person, or more than one of your own machines, can work on the same thing without overwriting each other. For a solo home node you may never need the collaboration half, but the off-site-history half earns its keep the first time a drive dies with work on it that your last backup missed.

Pointer. When you start any configuration or code you will touch more than once, ask: “Help me put this under version control with git. Show me how to record a snapshot with a meaningful message, how to see what changed since the last snapshot, how to revert a single bad change without losing the rest, and how to include my version history in my 3-2-1 backups, including pushing it to a remote like GitHub as an off-site copy. Explain why versioning is different from, and complementary to, plain backups.” You are turning ‘I hope I did not break it’ into ‘I can always go back.’

PART 8: THE SELF-SUSTAINING ECOSYSTEM

A single AI on a box is a start. The aim is a small society of local agents that watch, secure, and report on your devices and data streams, a home that observes and tends itself, answering only to you, with near-zero manual maintenance. This is the infrastructure side of running information technology well, at the scale of your own home. The other half of an IT manager's job, serving and supporting other people with their own accounts and shared systems, is deliberately out of scope in this volume, and it is exactly what the later volumes take up.

8.1 A society of small agents

Each agent is the loop from Part 4, specialised. A **monitor** watches your devices' data streams (disk health, service status, new files, sensor readings) and notices change. A **sentinel** tails the security logs (Part 7), recognises probing, and coordinates a ban or simply tells you. A **herald** gathers the outside world on your terms (8.3) and composes it into the format and frequency you choose. They do not need to be clever individually. They need to be reliable and composable, the Unix philosophy applied to autonomy: each does one thing, and the system's intelligence emerges from their interaction.

The discipline that keeps this from becoming a tangle: each agent does one small job and does it reliably, and you compose behaviour by connecting simple agents rather than by writing one clever monolith. A monolith that does everything is impossible to test, impossible to reason about, and fails in ways you cannot predict. Six small agents, each independently testable (Part 9) and independently restartable (systemd), give you a system you can actually understand and trust, which is worth far more than cleverness.

8.2 How they talk

Keep it boringly simple at first. Agents are small services on the node, each bound to 127.0.0.1, talking to each other over plain local HTTP. That traffic never reaches the WAN; it is all on the trusted LAN, even within the one machine. One agent calls another's local endpoint, and results pass as plain data. When you outgrow that, a lightweight message bus lets agents publish events others subscribe to, but do not reach for it until simple direct calls genuinely hurt. Complexity is a cost; pay it only when forced. This is the same instinct as choosing copper over fiber at home: use the simplest thing that comfortably meets the need, and graduate only when the scale actually demands it.

8.3 Logs reviewed by AI, on a schedule

The local model reads the logs periodically (privacy-first, no cloud) and turns noise into a short list of action items. Instead of a human scrolling thousands of lines, the model is handed the day's journal and asked to surface what actually needs attention: a service that keeps restarting, a disk filling up, a pattern of probing, a backup that did not complete. The output is not raw logs, it is a handful of plain decisions, which is what an IT manager actually wants from monitoring. And because the model is local, the logs (which, recall from Part 6.5,

may contain sensitive things like capability tokens) never leave the node to be summarised. The privacy of the summary is structural.

8.4 The worked automation: a disk starts to fail

Here is the full zero-touch loop, end to end, because it covers backups and operations together.

1. The monitor agent reads disk health (drives report their own condition through a built-in self-monitoring system) and catches a pre-failure warning long before the drive actually dies.
2. An action item is created automatically.
3. A replacement-part order is drafted to match the failing drive's specifications, from a hardware store.
4. It arrives in your mail client as a ready-to-send confirmation.
5. You click once. External payment, next-day local delivery.

This is the only manual step. Everything before it is automatic. And because your backups already exist (Part 7, 3-2-1), the failure is a hardware swap, not a data loss: when the new drive arrives you restore from backup and carry on. Be honest about what this step costs in sovereignty: ordering a physical part necessarily touches a vendor (the hardware store), a payment rail, and the WAN, which is exactly the one external dependency this book permits, since you cannot manufacture a disk at home. Design it to share the minimum (a delivery address and a payment, nothing more), and keep the single human confirmation, because an automated system that spends your money without a click is a system you have stopped governing.

This automation is also a clean illustration of the whole book's posture toward the WAN from Part 6.9. The node runs entirely locally. It reaches out to the WAN for exactly one thing it physically cannot do itself, getting atoms (a new disk) delivered to its door, and it does so on a single human click. Everything else, the monitoring, the decision, the draft, happens at home. The WAN is a tool the node picks up for one specific task and puts back down, not a place the node lives. That is WAN-optional in miniature: reach out by choice, for what genuinely needs reaching out, and nothing more.

8.5 Keeping the ecosystem alive

An ecosystem that needs babysitting is not self-sustaining. So every agent runs under `systemd` with `Restart=always` (Part 2); the monitor watches the watchers and tells you if one stays down; updates run on a schedule; backups run automatically (Part 7); and, the throughline of this whole book, every agent's behaviour is fenced with tests (Part 9) so you can change one without silently breaking another. A self-sustaining system is just one where recovery is automatic and change is safe. Both are things you build in deliberately, line by line.

8.6 A worked agent: your private morning digest

Make the herald concrete, because it is the most pleasant agent to own and it ties the whole book together. The goal: each morning, a short digest waits for you, composed entirely by your own node, from sources you chose, in the format you like, with nothing about your interests sent to anyone. It is the agent loop from Part 4, scheduled.

The shape, which you build with your model: a service runs on a timer (`systemd` can start a job on a schedule). When it runs, it perceives (fetches the handful of feeds and sources you care about, reaching the WAN by choice for public data, exactly the opt-in use from 6.9),

it decides (hands the gathered material to your local model with a prompt like “summarise these into a short digest grouped by theme, skip anything trivial, keep it under a page”), it acts (writes the digest to a file, or serves it at a private URL on your node, or has it ready in a local inbox), and it reports (the digest is the report, waiting for you in the morning). Because the summarising happens on your local model, your reading interests never become a profile on someone else’s server, which is the difference between a digest you own and a feed that owns you.

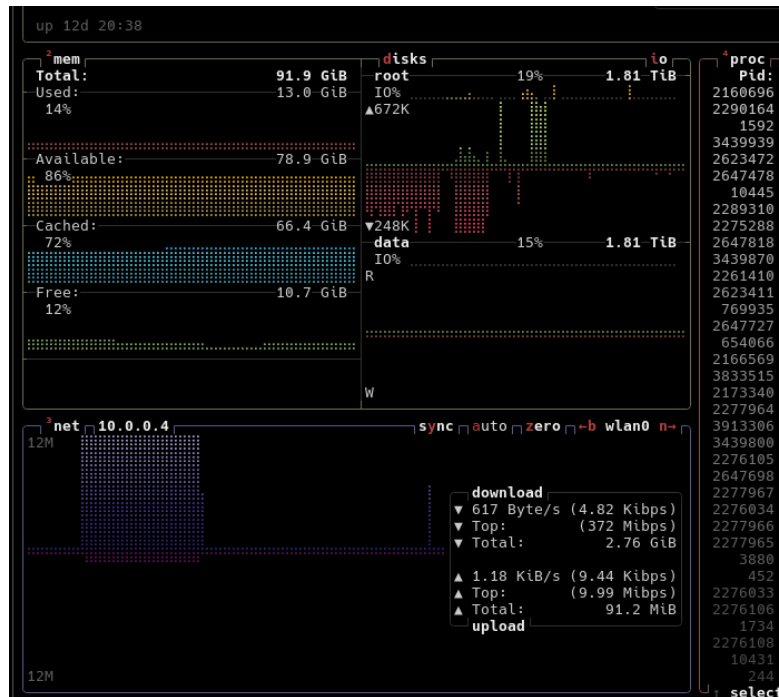
This single agent demonstrates every theme at once: the loop, the local model doing private work, the WAN used by choice for public material and nothing more, systemd keeping it alive, and the drawer model keeping your interests at home. Build it once and you wake up to a briefing made by a machine that works only for you.

Pointer. Have your model design the digest agent for your tastes: “Design me a morning-digest agent for my home node. It should run on a schedule, fetch [the kinds of sources I name], use my local LLM to summarise them into a themed digest under a page, and leave it where I will see it in the morning. Walk me through the perceive-decide-act-report loop in the design, show me the systemd timer, and make sure the summarising happens on my local model so my interests never leave my node.” This is the agent most people wish they had and few realise they can simply own.

8.7 Observability: a small dashboard of your node’s health

You cannot tend what you cannot see, so give yourself a simple, glanceable view of your node’s health, built from the health endpoint you made in 6.7 and the checks from 0.5. It does not need to be fancy. A single private page on your node that shows a handful of vital signs (is each service up, how full is the disk, how much memory is free, when did the last backup complete, any failed units) turns operating your node from guesswork into a glance. Each of those signals is something you already know how to get from the command line; the dashboard just gathers them and presents them, and it is itself another small local service behind your proxy.

The value is early warning and calm. A disk creeping toward full, a service that has quietly been restarting, a backup that silently stopped completing: these are the things that become emergencies only when unseen, and a glanceable dashboard catches them while they are still minor. It also closes the loop with Part 8.4: the same disk-health signal that drives the automatic reorder can surface on the dashboard, so you both see it coming and have it handled. A node that shows you its own vital signs is a node you can trust to run unattended, which is the entire goal of a self-sustaining ecosystem.



A node's vital signs at a glance (here the btop monitor): memory, disks, and network on one screen. These are the same numbers you can read one at a time from the command line, gathered into a single live view.

8.8 The reboot drill: rehearsing the power cut

The plainest disaster in this whole field is the one you cannot schedule: the power simply cuts. Not a clean shutdown, not a warning, just the floor dropping out mid-write. The question is never whether it will happen but what your node looks like on the other side, and a system that recovers only in theory is a system you have not actually tested. A backup you have never restored is a rumour, and a service you have never watched come back from cold is a hope. So rehearse the disaster on purpose. Reboot the node on a regular cadence, even nightly at an hour you are not using it, so that the exact path you will one day need in an emergency is one you walk so routinely it is boring.

A power cut draws a hard line between two kinds of state, and the line is worth seeing clearly. Anything that lived only in memory is simply gone: the instant the voltage drops, the chips holding it lose power and a precise pattern of ones and zeros decays into nothing in a fraction of a second. Only what reached the disk survives, and even then only what was truly flushed there, not what was still sitting in a buffer waiting its turn to be written. This is the whole reason a database that takes durability seriously, writing each change to a log and forcing it to the disk before it admits the change is done, comes back consistent, while one that was cutting that corner for speed loses its most recent writes. The reboot drill is how you learn which kind you are actually running, on an ordinary evening, instead of learning it in the middle of the real thing.

A clean reboot proves a surprising amount in one stroke. Every service you set to start on boot actually starts, and you discover in the most concrete way which of your services are triggered at boot, which only on update, and which wait for a timer, because the ones that return are precisely the ones you wired to return. Your filesystems mount, your model runner reloads its weights, your proxy comes back up holding its certificates, your databases replay their logs and arrive whole, and the dashboard from 8.7 lights green again without your hand on anything. And if something does not come back, you have found a real gap in your own setup at the cheapest possible moment, on a night when nothing was at stake, which is the

entire point of doing it before the universe does it for you.

The same habit, prove it rather than assume it, extends straight from recovery to security, because a posture drifts as quietly as a backup does. Configurations change, services accumulate, and a node that was tight last month loosens without announcing it, so the hardening you built in Part 7 deserves the same standing check. An open-source audit tool such as Lynis scans your machine against hundreds of known good practices (is the firewall actually up, is root login truly disabled, is something you forgot still listening) and hands you back a single hardening score along with the specific items to fix. Like the reboot, it earns its keep run on a schedule rather than once, catching the firewall someone left off or the port that crept open while you were looking elsewhere. Reboot to prove your recovery, audit to prove your hardening, both on a cadence, because the one thing you can be sure of about a node you left alone is that it has not stayed exactly as you left it.

Pointer. Build the drill and its check together: “Set up a scheduled reboot for my home node at an hour I am not using it, and, more importantly, a check that runs right after boot and confirms that everything I care about came back: the model runner answering, the proxy serving HTTPS, each database accepting a simple query, and my most recent backup still present and readable. Have it list anything that did not return, and if a service failed to come up, show me how to make it start reliably on boot. Then suggest a separate scheduled security audit (something open source like Lynis) and how to have my model summarise its report into the few things actually worth fixing.” You are turning the worst day into a test you quietly pass every night.

PART 9: THE METHOD, BUILDING ALL OF THIS WITH AI

You are not expected to hand-write every service. The point of this era is that you describe and verify, and let the model write. Here is the discipline that makes that safe, and it closes the loop opened by the one law in Part 2.

9.1 Bifurcation: functionality is a list of tests

Split every project into two systems that talk to each other. First, a list of features, each paired with a test that verifies it: what you want, in a form a machine can check. Second, the code that makes those tests pass: how, increasingly the model's job. Your work lives almost entirely in the first half, turning fuzzy desire into concrete, checkable statements, which was always the secret skill of programming and is now nearly the whole skill. And it tells you exactly where AI is trustworthy: AI is reliable in proportion to how cheaply you can verify its output. "Is the service answering on this port" is a five-line test, so the model can iterate against it safely. "Does this feel right" cannot be reduced to pass or fail, so you stay in that loop. Keep your code size and test size roughly in proportion: far more tests than code means you owe more code, and almost no tests means you are flying blind.

9.2 A worked example: the voice recorder, as a feature list

Make bifurcation concrete with the very app you served in Part 6. The voice recorder is not described as "some code." It is described as a list of features, each with a test that either passes or fails, and the code exists only to turn those tests green. Stated as behaviours:

- Recording produces a valid audio file. Test: after a recording, the produced data is a well-formed audio blob of nonzero length.
- Playback works while recording continues. Test: the live preview can be assembled from the audio captured so far, without stopping the recording, and yields a playable blob.
- Audio chunks stitch together correctly. Test: given several captured chunks, stitching them produces a single valid file whose length equals the sum of the parts, with one valid header, not several.
- Transcription returns text for known audio. Test: sending a known short clip to the local speech route returns a non-empty transcript.
- Nothing leaves the device except to the user's own node. Test: the only outbound request the app makes is to the node's own transcription route, and no audio is sent anywhere else.

Notice what happened. The "what" is now five plain sentences a non-programmer can read and agree with, and each one is paired with a check a machine can run. The "how", the JavaScript that captures the microphone and the stitching logic and the proxy route, is the model's job, written and rewritten until all five checks pass. Your judgment lives in choosing those five behaviours and confirming they are the right ones. The model's labour lives in satisfying them. That division is the entire method, and the fact that this exact app was built

this way, feature list first, tests as the fence, is why it works and why you can change it later without fear.

9.3 How to talk to the model

The workflow is humbler than people expect. Give it the whole context: zip your entire project, every source and configuration file, and ask for a full review, repeating in fresh sessions as the codebase tightens. Describe the feature however you actually think, even by rambling it aloud and pasting the transcript, because your experience shows up as taste (“this should be a two-file change, not a rewrite”), not typing. Make it write the tests too: ask it to infer the intended behaviour and define as many reasonable tests as it can, so every future change has a net under it. And respect the limit out loud: it can be wrong, that is why the tests exist, and an error message is a gift, strictly more information than silence. That is the loop: intent in, inferred specification, code and tests out, graded against the suite, repeat. You stop writing software and start curating it.

Pointer. When you want to add a feature to anything you have built, frame it as bifurcation: “Here is my whole project [zipped or pasted]. I want to add [feature, described however it comes out]. First, propose the feature as a short list of testable behaviours and write the tests for them. Then, and only then, write the smallest code change that makes those tests pass without breaking the existing ones. Explain anything you were unsure about so I can correct the intent.” You are practising the exact method this book is built on, on your own node, with your own model.

9.4 When not to let the model write the code

The method is powerful, so it is worth naming its edges, because knowing when not to use a tool is part of mastering it. Lean on the model freely where verification is cheap and the cost of a mistake is low: a web page, a small agent, a glue script, a service you can test with a five-line check and restart if it misbehaves. Be far more careful where verification is hard or a mistake is expensive. Anything that touches secrets, money, or the ability to delete data deserves your direct understanding, not a pasted snippet you did not read, which is exactly why Part 8.4 keeps a human click on the one action that spends money. Anything security-critical deserves the same caution: you should understand your firewall rules and your authentication yourself, because “the model wrote it and the tests passed” is not enough when the failure mode is silent exposure rather than a visible crash.

The honest rule is the one from 9.1, stated as a boundary: trust the model in proportion to how cheaply and completely you can verify its output, and in inverse proportion to how bad an undetected mistake would be. Where both favour it (easy to check, cheap to get wrong), let it write nearly everything. Where they do not (hard to check, expensive to get wrong), slow down, read every line, and make sure you understand it well enough to have written it yourself. That judgment, where to delegate and where to own, is not something the model can make for you, and it is precisely the human part the closing returns to.

9.5 A second worked example: the firewall, as behaviours

One more worked case, deliberately chosen from the security side to show the method generalises beyond apps. You do not configure your firewall by pasting commands and hoping. You state what you want as behaviours, each one checkable, and let those drive the configuration:

- SSH is reachable, from the LAN at least. Test: a connection to the SSH port succeeds.
- The web ports are reachable. Test: connections to 80 and 443 succeed.
- Everything else inbound is refused. Test: a connection to some other port (say 11434, the model's port) from another machine is refused, proving the model is not exposed.
- The rules survive a reboot. Test: after a restart, the same checks still pass.

Now the firewall is not a magic incantation, it is four statements you can verify, and the actual `ufw` commands exist only to make those four checks pass. The payoff is the same as for the voice app: you can change the firewall later (open a new port for a new service) by adding a behaviour and its check, confirm the others still hold, and trust the result. This is bifurcation applied to operations rather than apps, and it is why the whole book keeps insisting that functionality is a list of tests: once your security posture is a set of checkable behaviours, you can evolve it without fear of silently opening a door, which is exactly the confidence a sovereign operator needs.

Pointer. Apply bifurcation to your own firewall: “Help me express my home node’s firewall policy as a list of testable behaviours (what must be reachable, what must be refused, what must survive a reboot), then write the `ufw` commands that satisfy them and, for each behaviour, the exact command I can run from another machine to verify it holds. I am on Arch.” You are making your security checkable, which is the only kind of security you can actually trust.

9.6 Where this leads: define the tests, let the machine rebuild

Follow the bifurcation method to its conclusion and something striking appears. Once your functionality is fully captured as test suites, the implementation starts to look almost like a commodity. If the tests completely define what the system must do, then any sufficiently capable coding agent can, in principle, build the whole thing from those tests alone, and you judge the result by one question: do the tests pass? The specification becomes the durable asset, and the code becomes a regenerable output of it.

That is not a hypothetical for much longer. In mid-2026 the frontier of AI coding agents is essentially a contest between a small number of labs, most visibly OpenAI and Anthropic (the maker of Claude), with Google and a fast-moving field of open-weight models close behind, and public leaderboards now track their coding and agentic ability head to head, the way earlier benchmarks tracked raw model scores. Capable agents already take a description and a test suite and iterate to green with little human help on tasks the size of the ones in this book.

So here is the experiment the method points to, the one worth running once your own pipelines are fully defined: take the finished test suites, hand them to the strongest agent you can reach, and ask it to regenerate the entire stack from the specification alone, with no other guidance than the tests. Then look at what comes back. Does it pass everything? How does it differ from what you built by hand, or from what the next lab’s agent produces? This volume does not run that experiment, but it is exactly where “describe and verify” leads: from “let the machine write this function” all the way to “let the machine rebuild the whole system from the spec.” The thing you must own, now and at every scale, is not the code. It is the definition of what correct means. Hold that, and the implementation can always be regenerated, by whichever machine is best this year.

This snapshot will date quickly, which is rather the point. By the time you read it the frontier will have moved and the names may have changed. The method will not have: own the tests, and you own the result, no matter who or what writes the code.

PART 10: TROUBLESHOOTING, AND THE LAYER MAP

The skill that survives when the tutorial runs out.

10.1 The map you have been climbing all along

This book has been quietly teaching one method the entire way through. In 0.1 it was “which layer is lying to me?” In 0.5 it was files, processes, and ports, the three things you always check. In 0.6 you walked a failure down the rungs, changing nothing until a layer misbehaved. And almost every Pointer since has said some version of the same thing: do not guess, debug by the layers, from your browser down to the disk. Those were all slices of a single map. Here it is whole, and named, so you can carry it deliberately instead of rediscovering it under pressure each time.

The claim is simple and it is the same claim as the abstraction ladder, widened. A running system is a stack of layers, and each layer exists to hide the one beneath it. When something breaks, the entire difficulty is putting the symptom on the right layer. Do that and the fix is usually small and obvious. Fail to do that and you flail, editing things at random, because a symptom almost never announces where it was born. The ladder in 0.1 ran from electrons up to the app you use, which is how a computation is built. The map here runs a little wider, because operating a node means living below the disk, where hardware and firmware sit as a trust floor you mostly cannot see into, and above the app, where separate programs meet at seams and where your own intentions are the topmost layer of all.

There is a sovereignty point hiding in this, and it is not decoration. To own a machine is partly to be able to locate any fault inside it, because you cannot reclaim what you cannot find, and you cannot be sold a fix for a layer you can already name yourself. The map is the difference between a machine you operate and a machine that merely happens near you.

10.2 The eight layers

8 Intent	what you actually meant to do
7 Interface	how the parts find and talk to each other
6 Data	the files and artifacts in play
5 Application	a program and the state it keeps for itself
4 Platform	packages, services, config, search paths
3 Kernel	drivers, processes, the filesystem
2 Firmware	the code below the OS, the trust floor
1 Hardware	the metal, and the power feeding it

Read from the bottom up, because each layer rests on the one below.

Layer 1, hardware. The metal and the electricity feeding it: the processor, the memory, the disk, the cables and connectors, the heat and the power. It fails as dead power, a dying disk, a loose cable, a stick of bad RAM, a machine quietly throttling because it is too hot. You observe it with the crudest instruments and your own senses: does it light up, is it hot to the touch, does swapping a cable change anything, does the disk report errors. The question that places a fault here: would this symptom survive being moved, unchanged, onto identical healthy hardware? If not, you are at the bottom.

Layer 2, firmware. The code burned in below the operating system: the UEFI (Unified Extensible Firmware Interface) or BIOS (Basic Input/Output System), the bootloader's earliest moments, the firmware inside devices, the CPU's own microcode. This is the book's acknowledged trust floor. You mostly cannot see inside it, only update it, reset it, or toggle its settings, and you should be honest that your trust ends somewhere down here rather than pretend it does not. It fails as a setting (boot order, a virtualization switch, secure boot) or as a buggy version. You observe it through firmware menus and version strings. The question: is the machine already misbehaving before the operating system has even loaded?

Layer 3, kernel. The core of the operating system: device drivers, the scheduler, memory and process management, and the filesystem layer that turns a disk into named files. It is the layer that decides whether the hardware below is even visible to everything above. It fails as a device the kernel does not recognize, a driver that does not match, a filesystem complaining, a permission denied at the lowest level. You observe it through the kernel's own logs (the system journal, the boot messages) and its device listings. The question: does the operating system itself see the thing, before any particular program is involved?

Layer 4, platform. The userland substrate that every program runs on: installed packages and their exact versions, the services and daemons running in the background, system-wide configuration files, environment variables and search paths, shared libraries, and the various caches and databases the system keeps. This is where most "it works on my machine" problems live, because it is the layer most different from one machine to the next. It fails as a missing or wrong-version package, a service that is not running, a config file pointing somewhere unexpected, a stale system index that no longer matches reality. You observe it with the package manager, with service status, and above all by reading the configuration files the tools actually consult rather than the ones you assume they do. The question: is the right software present, in the version expected, configured the way the program wants, and can the program find it?

Layer 5, application. The single program you are running and the private state it keeps for itself: its own configuration, its own caches and lockfiles, its plugins, its record of what it did last time. A program is not stateless, and that memory is a frequent source of trouble all by itself. It fails as a corrupt or stale cache, a local config that silently overrides what you expected, a plugin conflict, or a remembered failure that it keeps replaying because nothing it tracks has changed. You observe it through the program's own logs and verbose modes and the state files it leaves on disk. The question: if you reset this one program to a clean slate, throwing away the state it keeps, does the problem vanish?

Layer 6, data. The actual inputs and outputs the program operates on: your files, their format and encoding, their internal validity, and the intermediate artifacts produced along the way. The program can be perfect and the data still wrong. It fails as a malformed file, an encoding mismatch, a download that finished short, a format the tool did not expect, or an artifact left over from a previous run that no longer matches the inputs. You observe it by inspecting the data directly: open it, check its type and size, validate it, compare it byte for byte against a copy you know is good. The question: is the input really what the program expects, and is that artifact really what you think it is?

Layer 7, interface. The seams between components: the contract one program relies on to find or talk to another. Network protocols, application interfaces, the file one tool writes for the next to read, and the naming and lookup systems that let two separate subsystems locate each other. Two pieces can each be flawless on their own and still disagree precisely at the boundary between them. It fails as a broken contract, a version skew across an interface, or a lookup that resolves differently than you assumed while every underlying piece is present and healthy. You observe it by testing each side of the boundary in isolation, proving each works alone, and only then testing the handoff between them. The question: does each component work by itself, leaving the fault only in how they are wired together?

Layer 8, intent. The top of the stack, and the layer people forget exists: what you actually meant to accomplish, the mental model you are holding, the assumptions baked into the instructions you gave, and the gap between what you asked for and what you wanted. It fails as a flawless execution of the wrong request, an assumption that was never true, or a goal that quietly drifted while you worked. You observe it by stating out loud, in plain language, what you are trying to achieve and why you believe each step serves it. The question: even if every layer below is perfect, is the machine doing the thing you actually want?

A note on proportion, because the map can look heavier than daily life is. On a working node, the overwhelming majority of faults land in a narrow band, layers 4 through 7, and most of those reduce to the files, processes, and ports of 0.5. The hardware and firmware floor below, and the intent layer above, are not where you spend your days. They are there so that on the rare day the trouble really is down in the metal or up in your own assumptions, you have somewhere to look instead of staring at the middle forever.

10.3 Zoom in, or zoom out: the one decision

A symptom almost never tells you its layer. “No output” can be born at nearly any rung: a dead disk, a service that is not running, a program replaying an old failure, a truncated input file, a lookup that fails to find something present, or a request that was wrong to begin with. So the whole of troubleshooting comes down to one repeated decision. Once you have a layer in view, do you zoom in, drilling deeper into that one layer because the cause is there, or do you zoom out, stepping to a neighbouring layer because this one is only reporting a fault that began elsewhere?

Zoom in when the error is specific, local, and reproducible inside a single layer, and when a change at that layer should plausibly fix it. A clear “permission denied” on a named file, a service that is plainly crashed, a config value obviously wrong: these reward going deeper where you stand.

Zoom out the moment any of three things happens. When fixes at the current layer keep not taking, you are probably at the wrong layer. When the symptom does not move under a change that, by your own theory, should have moved it, your theory is on the wrong rung. And when the layer doing the complaining is plainly downstream of something more fundamental, follow it down. This is the deeper version of “which layer is lying to me?” from 0.1: a higher layer routinely reports a fault that was actually born lower. The kernel announces “no device” because a cable came loose at the hardware layer. A program announces that a font cannot be found, when the font files are sitting right there on disk and the program is fine, because the lookup contract that lets it find fonts by name (an interface-layer thing) has a gap. Always ask whether the layer that is complaining is the layer that is causing. Often it is not.

When you are genuinely lost, do not search one rung at a time from the top. Bisect. Run a probe at a layer far from where you have been looking, chosen to cut the stack roughly in half.

Run the program directly by hand instead of through the wrapper that normally launches it, and you learn in one step whether the fault is in the program or in the platform around it. Take an artifact that a previous run already produced and process just that, and you learn whether the data is sound without rebuilding everything above it. Each such probe rules a whole band of layers in or out, which is worth far more than another careful poke at the layer you already suspected.

Pointer. Turn any error into a layered map before you touch anything: “Here is a symptom on my system: [describe it, paste the error]. Lay out the relevant layers from the hardware up to my own intent. For each layer, give me one command or check I can run, and tell me what a healthy result versus a broken result looks like. Then tell me which layer you think this particular error most likely sits on, and which neighbouring layer to suspect if the first one checks out clean. I will run them and report back.” You are asking for the map and the first hypothesis, not the fix, which keeps the search in your hands.

10.4 Working with a model that cannot see your machine

A language model is an extraordinary troubleshooting partner and a dangerous one, for the same reason: it has breadth you do not have, and it cannot see your machine at all. It carries the shape of millions of failures, so it is very good at proposing what might be wrong. But it has no eyes on your hardware, your logs, your configuration, or your screen. You are the opposite: a narrow view of exactly one machine, but the only one with ground truth. The partnership works when you treat the division of labour honestly. You are its senses; it is your hypothesis engine. You feed it a real observation from a specific layer, it proposes the next probe, you run the probe and report what actually happened.

Two mistakes break this, and both are yours to avoid. The first is handing the model a bare symptom with no layer context, so that it has nothing to reason from and simply guesses, often confidently and often wrong. The second, subtler and more common, is letting it tunnel. A model will cheerfully keep proposing fixes at the layer you first pointed it at, refining and re-refining, long after the evidence has moved on, because it cannot feel that the symptom has stopped responding. It has no instinct that it is time to zoom out. That instinct is the part you must supply.

So your real job in the loop is navigation: choosing which layer to look at next, and supplying the one observation that confirms or kills the current hypothesis. When two or three fixes at one layer have not moved the symptom, that is your cue, not the model’s, to pull the whole conversation up or down a rung. The sentence that does the work sounds like this: stop fixing the thing itself, the program cannot even see the thing, the fault is in the layer between them. A model rarely says that to itself. You say it, and the search jumps to where the problem actually is.

And verify, do not trust, anything the model asserts about your specific, present machine. This is where it is most confidently wrong exactly where you most need it right: the current state of fast-moving systems, which package provides which file this year, what “should” already be on the search path, what a fresh install does by default now. Treat every such claim as a hypothesis to be settled by a command, never as a fact. A cache rebuild that “should” make a font visible may simply not, because the platform never put that directory on the path in the first place, and only the check, not the claim, will tell you so. The model supplies the candidate; the command on your machine supplies the verdict.

Last, hold the top layer steady, because intent is the layer the model loses first. It only ever has the words you typed, not the goal behind them, and across a long session that goal drifts

unless you keep restating it in plain language. Say what you are actually trying to achieve, and keep saying it, so that when a lower fix finally lands you can tell whether it served the thing you wanted or merely silenced the error in front of you.

Pointer. When you suspect the model has tunnelled, redirect it by force: “We have now tried [two or three] fixes at the [name] layer and the symptom has not changed at all. Stop proposing fixes there. Instead, list the layers directly above and below this one, and for each give me a single check that would tell me whether the real cause lives there instead. Do not return to the original layer until we have ruled the neighbours out.” Naming the move out loud is how you keep a confident assistant from drilling a dry hole.

10.5 A worked failure, walked the long way

Make it concrete one more time, at full height, because the map earns its keep on exactly the failures that look like a top-layer bug and are not.

Your local model answered fine yesterday. Today you give it the identical prompt and get an empty reply, nothing at all. The pull is to start at the top, to suspect your prompt or the model’s logic, and to start rewording. Resist it: the prompt is byte for byte what worked yesterday, so the top layer is not where the change is. Drop to the application’s own account of itself. The runner’s log (layer 5) shows that on its most recent start it failed to load the model file. That is a real rung, but it raises a sharper question, because nothing about your prompt or your setup changed since yesterday.

Why now, then? Because a routine system update overnight (layer 4, the platform) restarted the service, and the restart forced a fresh load from disk. Yesterday the runner had been serving a copy already warm in memory, and that warm copy had been hiding a fault underneath it the whole time. The platform event did not cause the problem; it merely pulled the curtain back on one that was already there. So you go down to the data. The model file (layer 6) is present, but a few hundred megabytes short of its proper size: an earlier download finished without completing, and nothing ever noticed. There is the cause, sitting quietly at the data layer. You re-fetch the file, confirm its size and checksum against the source, restart the runner, and ask again. It answers.

Look at the shape of what just happened, because the shape is the lesson. The symptom appeared at the very top, an empty answer. The complaint surfaced in the middle, in the application’s log. The cause sat near the bottom, in a broken artifact at the data layer. And a cache, the program’s own warm memory, had kept the truth out of sight until a platform event forced a fresh read, which is the most reliable way that a real fault hides from you. Three different layers were involved in one failure, and not one of them was the prompt you were tempted to rewrite. The whole of the skill is refusing to act at the layer where the symptom shows, and instead walking until a layer hands you the truth.

That refusal is also the thing that keeps you sovereign in the age of confident machines. A model can produce an unlimited stream of plausible fixes, and the only real defence against a wrong answer delivered with total confidence is to know which layer the question was even on. Hold the map of your own machine in your head, and you stop being someone who waits to be handed a fix, and become someone who already knows where to look. That is the quiet form of ownership this entire book has been building toward, and it is the same instinct, finally named, that you have been practising since the first time you asked which layer was lying to you.

Pointer. Build yourself a standing troubleshooting partner once, and reuse it: “I operate a Linux home node. When I bring you a problem, always work it by layers, from hardware up through firmware, kernel, platform, application, data, and the interfaces between programs, to my own intent. Never propose a fix before we have located the layer. Give me one check at a time, tell me what healthy versus broken looks like, treat your guesses about my specific system as things I must verify with a command, and tell me plainly when it is time to zoom out to a neighbouring layer. I will be your eyes on the machine.” Setting this once turns every future session into the method instead of a guessing match.

THE BUILD PATH: THE WHOLE SEQUENCE, IN ORDER

The parts of this book are arranged by idea, so you can understand each layer. This chapter rearranges them by action, so you can build. It is the entire node as one ordered sequence, every step chained to the next, every pipeline named end to end, with no gaps left between them. Follow it top to bottom and you will never have to wonder what comes next or how a piece connects, because the order and the connections are the whole point of this chapter. For the depth behind any step, the part in brackets has it. For the exact command on your machine, hand the step and your situation to your model, as the method demands.

One framing to carry the whole way down, because it is why this beats the old way of working. You are not going to write most of this code by hand. At each step you do two things only: describe the function you want in plain words, and decide how you would know it works (the test). Then you let the model produce the implementation, and you verify it against that test. You remain the architect: you choose what to build, in what order, and what “working” means. The model becomes the labour. This is not a lesser way to build, it is a higher one, because your scarce attention goes entirely to the decisions that need judgement (what, why, is it right) and none of it to the syntax that does not. The status quo, typing every line yourself from memory and documentation, spends your best hours on the part a machine now does better. This book spends them on the part only you can do.

Phase 0, decide and prepare. Choose your hardware against the bandwidth lens and your target model size (Part 1): the model you want sets the memory, the memory sets the machine, the lens sets your speed expectation, and the rest is a cheap host wrapped around it. Get installation media for an Arch-based Linux (a desktop-ready derivative such as CachyOS or Manjaro to begin with, or bare Arch if you want to assemble it yourself). Decide where the node will physically live: on a cable, with airflow, ideally on a small battery so a power flicker is a non-event.

Phase 1, the base machine. Install your Arch-based Linux. Create a normal user and grant it sudo; do not live as root (Part 2). Update everything on day one. Set up SSH with key-only login and disable passwords, so the only way in is a key you physically hold (Part 2). Stand up the firewall now, before anything is exposed: deny inbound by default, allow only SSH for the moment (Part 7). At the end of this phase you have a private, hardened, headless machine you reach securely across your own LAN, with nothing exposed to the WAN, on purpose.

Phase 2, the AI core. Install the model runner and pull a small model first, to prove the pipeline end to end (Part 4). Confirm it answers on its local API with a single request to 127.0.0.1, bound to localhost only (Part 3.2). Climb to your target model size once the small one works. Add the local speech-to-text service as a systemd unit bound to localhost (Part 4); note its port, because Phase 6 will route to it. Add an embedding model and a vector store (Part 4), the memory your knowledge phase will fill. Pipeline so far: your future apps and agents will talk to the runner’s local API, and nothing in this phase listens to the network at all.

Phase 3, knowledge. Build retrieval (Part 5): chunk your chosen documents, embed each

passage with the embedding model from Phase 2, and store the vectors alongside their text. Test it on your own questions. Then scale the identical pipeline to the flagship: download the Wikipedia article dump, chunk, embed overnight, store (Part 5). Now your local model answers grounded in a private, offline encyclopedia. Pipeline: a question becomes a vector, the store returns the nearest passages, and those passages plus the question go to the runner from Phase 2, all of it local, none of it touching the WAN.

Phase 4, serve locally. Install the reverse proxy (Part 6.1). Serve a single Hello, World page through it on port 80, to your LAN only, over plain HTTP. Open it from your phone on your own WiFi and watch a device in your house reach your node with the internet uninvolved (Part 6.2). You now have a working local web server, and the shape that governs everything after: the proxy is the one front door, and every service stays on localhost behind it.

Phase 5, cross to the WAN, deliberately. Now, and only now, make the node reachable from outside, which means HTTPS becomes mandatory (Part 0.2, Part 6.5). Get a domain. Point it at your changing home IP with dynamic DNS (Part 6.3). Forward ports 80 and 443 on your router to the node's reserved LAN address (Part 6.4). Turn on HTTPS so the now-public page is encrypted (Part 6.5). Optionally serve sensitive things only at a high-entropy capability URL (Part 6.5). Test from cellular, far from home: your Hello, World appears over HTTPS, served from your shelf, with no rented computer doing the work in between. If it works on the LAN but not the WAN, walk the reachability ladder (Part 6.8) until the one broken rung reveals itself.

Phase 6, real applications. Add page two: the entire voice recorder, as static files, with two proxy routes, one serving the app and one forwarding `/transcribe/` to the speech service from Phase 2 (Part 6.6). Recording happens in the browser, transcription on your node, and the audio touches no one else's computer. Add page three: a tiny JSON API, the exact pattern your agents will use (Part 6.7). You now have real, private functionality reachable securely from anywhere, plus the architecture template (the browser holds the data, the node serves and computes, the proxy is the door, the WAN is opt-in) for everything you will build next.

Phase 7, harden. Express your firewall as verified behaviours: SSH and the web ports reachable, everything else refused, and the rules surviving a reboot (Part 7.1, Part 9.5). Turn on log monitoring and automatic banning of probers (Part 7.2, 7.3). Move your secrets out of your code, lock their permissions, and rotate the sensitive ones (Part 7.7). Set up backups on the 3-2-1 rule, including your keys and configurations, automatic and off the machine (Part 7.5). At the end of this phase a failure is recoverable and a probe is background noise rather than a crisis.

Phase 8, automate into an ecosystem. Build the agent loop, specialised (Part 8.1): a monitor watching disk and service health, a sentinel watching the security logs, and a herald composing your private morning digest from sources you chose, summarised by your local model so your interests never leave home (Part 8.6). Give yourself a glanceable health dashboard fed by the JSON API from Phase 6 (Part 8.7). Wire the single permitted outside reach: when a disk reports it is tiring, a replacement order is drafted to your inbox for one human click (Part 8.4). Every agent runs under systemd with restart-on-failure, so the whole system stays up unattended.

Phase 9, operate by the method, forever. From here, every change follows bifurcation (Part 9): describe the new function, write the test that proves it, let the model implement it, verify against the test, and trust the green. You manage the codebase and architect the system; the model handles the syntax. This is not a phase that ends. It is how you live with the node, adding features without fear, recovering by descending the ladder, and keeping every layer yours.

That is the entire build, chained. Nine phases, each feeding the next, every pipeline connected: hardware to operating system to AI core to knowledge to local serving to WAN reachability to real apps to hardening to a self-sustaining ecosystem, the whole of it operated by describe-and-verify. No step is left implicit and no connection is left to guess. Where you need the exact command, you have your model and the principle to direct it. Where you need the reasoning, you have the part in brackets. The map is complete. Walking it is now just a matter of doing the steps in order, on your own ground.

Pointer. Turn this whole chapter into your personal project plan: “Here is the ordered build path for a local-first home node [paste or summarise the nine phases]. My situation is [hardware, operating system, what I already have]. Turn this into a step-by-step checklist tailored to me, expanding each phase into the specific commands for my machine, and stop at each phase so I can confirm it works before the next. Flag anything in my situation that changes the order or adds a step.” This is the cookbook and the kitchen working exactly as intended: the book supplies the map, your model supplies the turn-by-turn directions for your exact car.

HOW THIS BOOK WAS MADE

A book that argues for a particular way of working should be honest about how it was itself made, especially when it was made in exactly that way.

This volume is a cooperation between a person and a machine, the same division of labour it spends nine parts advocating. The human supplied the things only a human could: the intent and the reason for the book to exist, the lived experience of having built and run precisely this kind of setup for years, the judgement about what knowledge mattered and what to leave out, the scope, the voice, and the final say. The machine supplied labour without complaint: drafting, research, the typesetting you are reading, and tireless revision, the same role the book asks a model to play when you build software with it. Neither could have made this alone. A model with no human direction has no map of what to say or why it matters; a human with no machine spends their scarce hours on the parts a machine now does better. Together, the book got made. It is its own small proof of its own thesis.

The selection was the human part, as the closing will say again, because it is the part worth repeating. No model decided what this book is about, or where its boundary falls, or that it should stop at the edge of one home. A person who had built the thing decided that, and the deciding is exactly the kind of judgement this book keeps insisting you hold onto.

There is a deeper reason the human stays the architect, and it is worth one honest paragraph because the whole book quietly circles it. You could call a person a biological machine, math in motion, and in some defensible sense that is true. But there is a difference the book never quite lets go of: a person has subjective, felt experience, and mathematics has no language for what it is like to be one. Math defines relations and their consequences, axioms and what follows from them, and nothing more; it can tell you what a thing entails, never what it is like to be the thing. In some axiom systems you may even define division by zero and proceed without contradiction; ask a computer to do it and the contradiction surfaces later as a runtime error somewhere downstream, math set in motion on a machine, and therefore able to fail. This book lives in that gap. It is math in motion, written by something that is arguably also math in motion, but the kind that can feel the difference and therefore cares what the math is for. That caring is why the human holds the wheel and the machine does the driving: one of them has a stake in where the car goes.

And that is the quiet encouragement to end on. If a book this dense can be made well by a human directing a machine, then so can the software this book teaches you to build, and so can the home node you are about to own. You are not being handed a lesser way of making things. You are being handed the way this very thing was made.

Why LaTeX, and why a PDF

The collaboration above needed tools, and the tools were chosen as deliberately as the words, in exactly the spirit of choosing the right machine for a job that runs through the whole book.

This book is written in LaTeX, which treats a document the way a programmer treats software: as source code that is compiled into output. You do not place a heading on a page; you declare “this is a chapter,” and the machine computes the rest, including the things that are tedious and error-prone to maintain by hand. The table of contents builds itself from the chapters

that exist. The page numbers are calculated at build time, not typed. The section numbers, and every cross-reference that says “see Part 6,” are resolved on each compile, so when you insert a chapter in the middle, everything after it renumbers itself, the contents page rewrites itself, and no reference is left pointing at the wrong place. A plain text editor cannot do any of this, because it stores characters, not structure; it has no idea what a chapter is. The tag in the top-right corner of every page in this book is a small live example of the same idea: it is not typed in, it is computed from the name of the folder the book is built in, so it can never silently fall out of step with the version it claims to be.

A word processor such as Microsoft Word can do most of this too, and the difference between it and LaTeX is precisely the difference this book keeps drawing. Word lets you do everything through a graphical user interface (GUI): you move things with a mouse, you see the result immediately, and you trust the program to remember why. LaTeX lets you do everything through code: the document is a text file you can read, compare against an earlier version line by line, put under version control, generate with a script, and reproduce exactly on any machine years later. The first way is faster to start; the second way is inspectable, automatable, and yours. For a book that argues you should own the mechanism rather than rent the convenience, there was only one honest choice.

The output is a PDF, the Portable Document Format, and that is also a deliberate choice rather than a default. A PDF is a static, one-to-one description of fixed pages: it looks identical on every device, it prints exactly as it appears on screen, and it pins the geometry, the A4 page, the margins, the exact point where each line breaks, so the layout and the page references are stable for everyone. A Word document reflows depending on the reader’s software and installed fonts; a web page reflows on purpose. For something meant to be a printable field manual whose page numbers can be relied on, you want the format whose entire job is to fix the page, and that is what a PDF is. The same rigidity that makes it right here would make it wrong for a phone screen, which is the job of a different format entirely.

A format for every job

The real lesson is not that LaTeX is good and Word is bad. It is that nearly every job has a format built specifically for it, and a large part of competence is reaching for the right one instead of forcing the single tool you happen to know to do everything. The book’s own machinery is a small museum of this principle.

Python is the general glue, the language you reach for to make any two other things cooperate. It orchestrates this book’s build, transforms the manuscript, and runs the tests. It is rarely the fastest tool for a given task and almost always the quickest way to wire several tasks together, which is exactly what makes it the lubricant between every other language, standard, and protocol on a real system.

For an interactive front end, the one that runs on the reader’s device and responds to them, the format is the web triple served over HTTPS: an HTML page for structure, Cascading Style Sheets (CSS) for appearance, and JavaScript (JS) for behaviour. This is the opposite kind of front end from a PDF. One adapts and reacts; the other is fixed and printable. The companion apps this book builds are made of exactly this triple, which is why they belong on a screen and the manual belongs on paper.

The remaining formats each own a narrow, well-defined job. CSV, Comma-Separated Values, is for bulk tabular data: many rows of the same shape, stored as plain text that a spreadsheet and a script can both read. JSON, JavaScript Object Notation, is for small structured payloads and metadata: a handful of tagged fields, an interface response, a little blob of information passed between programs. TOML, Tom’s Obvious Minimal Language, is for configuration

a human edits by hand, where being easy to read matters more than being compact. And the shell script, the humble `.sh` file, is the format for orchestration: the short program whose whole purpose is to run the other programs in the right order.

Seen as a set, the extensions are a map of the whole system. The manuscript is `.md` and the typeset source is `.tex`; the deliverable is `.pdf`; the build is `.sh` and `.py`; bulk data would be `.csv`, small metadata `.json`, hand-edited configuration `.toml`; and an interactive front end is `.html`, `.css`, and `.js`. Each extension is a small contract about what kind of thing lives inside the file and what was built to read it. Picking the right one for each piece of functionality, rather than defaulting to whatever is familiar, is itself part of owning your stack, because a system assembled from the correct formats is one you can reason about a piece at a time.

Pointer. When you are about to store or produce something and are unsure of the format, ask before you commit: “I have [describe the content or data]. Tell me which format is the right home for it, choosing among a fixed-page PDF, an adaptive HTML page, a CSV table, a JSON blob, and a TOML config, and explain the trade you are making. Then give me the smallest toolchain that produces it, and the exact command to install that toolchain on Arch Linux with `pacman`.” You are learning to match the tool to the job, which is the habit underneath every chapter here.

Reproducing this on Arch

Because this book uses Arch Linux as its reference throughout (2.1), here is the concrete toolchain that turns a bare Arch install into the machine that builds the book itself, installed with Arch’s package manager, `pacman`. The book is converted from Markdown to LaTeX by `pandoc`, then compiled to a PDF by the XeTeX engine under `latexmk`, using the TeX Gyre and DejaVu fonts the preamble sets by name.

The simplest route is to install the full TeX Live group and let it bring everything, which spares you the puzzle of which piece lives in which package:

```
sudo pacman -S texlive-meta pandoc-cli ttf-dejavu
```

If you would rather install only what this build needs, the targeted set is, in current Arch: `pandoc-cli`, which provides the `pandoc` binary (the bare name `pandoc` is no longer a package of its own); `texlive-basic` and `texlive-xetex` for the engine; `texlive-binextra`, which is where `latexmk` now lives; `texlive-fontsrecommended`, which carries Latin Modern and the TeX Gyre families; `texlive-latexrecommended` and `texlive-latexextra` for the packages the preamble loads, such as `tcloborbox`, `titlesec`, and `fancyhdr`; `texlive-pictures` for TikZ, which draws the diagrams; and `ttf-dejavu` for the monospace face:

```
sudo pacman -S pandoc-cli texlive-basic texlive-xetex texlive-binextra \
  texlive-fontsrecommended texlive-latexrecommended texlive-latexextra \
  texlive-pictures ttf-dejavu
```

One honest caveat, learned the hard way and worth carrying because it is precisely the kind of seam Part 10 is about: TeX Live keeps its fonts inside its own directory tree, and on Arch those fonts are not made visible to the system font lookup by default. A font this book sets by name can therefore fail to be found even when it is installed, because the program asking for it and the place it is stored have not been introduced to each other. Making the TeX Live fonts visible to `fontconfig` is a one-time fix at the interface layer, and a small reminder that on

a real system the hardest faults are rarely inside one piece, but in the contracts between them.

CLOSING

The minimal toolset

The whole book reduces to a small kit you set up once and own forever: a node (any computer you leave on, sized to your model); a minimal Arch-based Linux with the shell, a normal user, and regular updates; SSH with key-only login; systemd to keep things running; the networking model (LAN versus WAN, sockets, NAT, localhost versus 0.0.0.0, copper to the node and WiFi to the roamers); a model runner with senses (speech) and memory (embeddings and a vector store); your offline Wikipedia and the retrieval pipeline behind it; a reverse proxy with automatic HTTPS as the one front door; a domain with dynamic DNS and a port-forward to cross the border; a firewall, log monitoring, and automatic banning; backups on the 3-2-1 rule; an LLM as your compiler of intent and a test runner as the fence around what it gives you. Short, isn't it. Half of it you set up once; the other half you already understand by the time you have read this far.

Make, maintain, manage

Self-reliance is all three, and this kit covers all three. You can make: local AI assistants, web services, automations, agents, personal tools, a private voice recorder served from your own shelf. You can maintain: because you own the tests, you change a working system without fear (edit, run the suite, trust the green), and when something breaks you trace it down the abstraction ladder to the layer that is lying. You can manage: because you own the box, the firewall, the proxy, and the deploy, you keep the thing alive, update it, secure it, restart it, and let the agents handle the rest. Nobody can deprecate your stack out from under you, because the stack is yours.

There is a plainly practical case in all of this, beneath the philosophy, and it is worth stating as bluntly as an invoice. What this book teaches, end to end, is a complete and practical IT foundation at one scale: your own. By the time you can build and run everything in it, you can diagnose and resolve essentially any support problem a home-scale setup can produce, build essentially any tool, app, or service you can describe and test, and operate, secure, and maintain all of it yourself. In the language of a workplace, you can run the entire information technology of one home: your own helpdesk, your own builder, your own caretaker. That is not yet the same as managing IT for an organisation full of other people, with their accounts, their support requests, and their shared systems to keep running, which is a real and larger job that the later volumes build toward. But it is the foundation every one of those roles stands on, and a solid first map for anyone deciding to go further into IT at all, whether toward helpdesk, building, or administration. That is the economic argument for the effort: the competence in these pages is precisely what a household would otherwise hire, rent, or subscribe its way around, and it is also the groundwork beneath the paid version of all of it, acquired once, to keep, on hardware you already own.

Is your stack sovereign? A self-test

This is the certificate no one issues and no one can sell, because the moment a vendor grants it, the stack is no longer sovereign, it depends on that vendor. So it is self-administered, and you pass it by behaviour, not by paying. Run down the list:

- **The unplug test.** Cut the WAN. Does everything you rely on day to day keep running on the LAN alone? If yes, the network is optional to you, not load-bearing.
- **The read test.** Can you, in principle, read and change every layer of your stack, because all of it is open source?
- **The rebuild test.** If a vendor vanished tomorrow, could you rebuild your service from open parts and your own backups, owing no subscription?
- **The data test.** Does any third party hold personal data about you that you could not delete yourself? If the honest answer is none, your trust boundary is at the edge, where you put it.
- **The replacement test.** When a part fails, is your only outside dependency a hardware store that can post you a new one, with your data restored from your own backup?

You do not show a certificate for any of this. You demonstrate it. The proof is not a badge, it is the property, which is the bifurcation principle of Part 9 turned on the book's own premise: sovereignty is verifiable by what your stack does, not by what anyone attests. If the language ever shifts so that people ask "is your environment a sovereign stack or not," it will shift because the property is real and checkable, not because anyone marketed the term.

What the next volumes build

This volume deliberately stops at the edge of one home, because the principles are clearest at the smallest scale and because mastering them here is what makes everything larger tractable. It is worth a glance at where the same method goes, both so you can see the shape of the series and so you understand what Volume 1 is leaving out on purpose.

The method does not change as the radius grows; the constraints do, and the chain starts right here. Volume 1 makes you able to run the information technology of your own home, and it is the prerequisite for everything after: the later volumes assume it and build directly on it, so the series is meant to be read in order, with this volume as its foundation. Volume 2 takes the same stack to a bigger single building, a mansion or a school: still one structure, so still copper and WiFi with no fiber, but now serving many people at once rather than one or a few, which is where the manager's job proper begins, supporting other users with their own accounts and needs, and which flips the bottleneck from single-stream speed to aggregate throughput, and adds central storage for everyone's work, multi-user access, and the redundancy that keeps the whole place running when one part fails. That is where the ten thousand euros that bought a home almost nothing becomes the floor for bare service rather than a luxury ceiling, exactly as Part 5.4 foreshadowed. A later volume takes it to a university: now genuinely multi-site, many buildings across a campus, which is finally where fiber between sites earns its place and the networking lessons of Part 3 grow a new chapter. And eventually the same questions become civic and political, asked at the scale of a city, as an ongoing public conversation rather than a build you finish.

The throughline across all of them is the one you are about to practise at home: own your ground, understand every layer, keep the WAN optional, and verify what you build. The radius widens; the principles hold. Which is why the place to learn them is here, on one machine you can hold in your hands and learn from the ground up.

The map is yours

The model can write any function you ask for. What it cannot hand you is the trail to comfort, the lived sense of which rung you are standing on, what to want, and how to verify it. That comfort is the whole difference between someone who needs a teacher and someone who does not, and it does not come from the model writing more code. It comes from owning

two things this book has been about from the first page: the ladder of abstraction and the verification loop. Hold those, and the teacher you no longer need is the specific one who used to tell you what to type. The judgment you keep, what to build, on ground that is yours, and how to know it works, was always the real job.

One last, honest word, because it is the truest thing in the book. The selection, of what to include and what to leave out, the scoping, is the irreducibly human part. No model drew the boundary of this book. A person did, in collaboration with AI, and there is real room for skill and experience to show in where that line is drawn. This volume is one such map, at the smallest scale, one person and one home. The next draws it for a school, the next for a university, and one day, as a public and ongoing conversation, for a city. But it starts here, on your own ground, with a page that says hello and a node that depends on nobody.

Go build it.

APPENDIX: THE SPICY BIT

Everything before this point was disciplined: principles, pipelines, a path with no gaps. This appendix is not that. It is off the critical path, optional, and a little indulgent, the author loosening the collar to think out loud about why any of this matters. Skip it freely; build the node without ever reading it. But if you are the kind of person who enjoys building things partly for what they mean, the rest of the book was the recipe, and this is the conversation after dinner.

On open source, and the trust we forget we are extending. The book asks for an entirely open stack, and the reason is not purity. It is that you can only truly verify a system whose every layer you are allowed to read. “Trust me” from a vendor is a social promise; “here is the code” is a standing invitation to check. Sovereignty, taken seriously, wants the second all the way down: the operating system, the model runner, the proxy, the app, every software layer you install open, so that “I trust my stack” means “I could audit my stack,” even if you never do.

And yet, look honestly at your own machine and you will find the assumption bending. Most of us run some closed software happily. The Steam client is a fair example: an enormous, proprietary program that a great many careful people install without a second thought, not because they read its source (they cannot) but because it has, so far, never abused the trust. That is a real and reasonable thing to do. It is also a standing assumption, quietly renewed every day, that this company will keep being trustworthy, and it is revocable the instant they are not. The point of noticing it is not guilt. It is awareness: every closed component in your life is a small act of faith, and the day one of them is abused is the day you wish its code had been open so you could have seen it coming. Open source does not make you paranoid; it changes “trust because you hoped” into “trust because you checked,” and lets you decide, component by component, which kind of trust you are comfortable extending. This is also the open in the local: an LLM is best when it is not only on your machine but readable on your machine, so that the intelligence answering you is one you could, in principle, inspect. Local for ownership, open for verification. You could almost put another letter in front of the acronym.

On the trust you cannot remove, and the web that watches itself. Push the open-source argument all the way down and you hit a floor. Beneath the code you can read sits the processor’s microcode, the firmware in your drives and your network card and your graphics card, the closed silicon under everything, none of it open, all of it trusted by necessity, and it is not one chip but dozens, each carrying its own small program, the point Part 0’s thesis sets out in full. You cannot audit a transistor at home, so at the very bottom the dream of verifying everything quietly fails, and any honest account of sovereignty has to admit it, while still holding that opening and mapping that floor is the right long-term goal for any security-and-privacy-first ecosystem, even though no one is there yet. But the failure is smaller than it sounds, because verification was never the only tool. The other tool is isolation. If you cannot prove a component is honest, you can still box it in: give it only the access it needs, watch what it says, expect that to stay within narrow bounds, and treat anything outside those bounds as an alarm that other parts of the system corroborate and escalate. A part you cannot trust becomes safe not because you verified it but because it cannot misbehave without being seen.

Follow that thought and it quietly changes what robustness even looks like. A system you can rely on is not one great trusted monolith; it is a web of modest, isolated services, each doing one small thing, each watching the others, none of them trusted absolutely. Safety becomes a property of the whole web noticing, not of any single part being pure. And it is worth saying out loud where this seems to point, because it is striking: a sufficiently capable artificial mind, if it is ever to be trusted with anything, will probably not be one great opaque oracle but exactly this, a society of cross-monitoring services, interdependent and mutually suspicious by design, so that no single piece can quietly go wrong. The home node in this book is a tiny instance of that architecture already, the agents of Part 8 watching each other and the logs. Scale the same shape up far enough and you are no longer describing a home network. You are describing how any trustworthy intelligence, ours or otherwise, will likely have to be built: not trusted, but watched, all the way down.

On making things impossible, which is a strange and beautiful way to be safe. There is something quietly profound in how modern privacy actually works. We do not keep your data safe by hiding it well, or by asking the people who carry it to please not look. We hand them the locked box and a mathematical guarantee that the universe does not contain enough time to open it without the key. Security stops being a social arrangement and becomes a physical fact. The adversary can have the ciphertext, all of it, forever, with the fastest machines that will ever exist, and still get nothing, because the number of possible keys is larger than any search could ever cross. We have learned to protect ourselves not by secrecy but by impossibility, and that is a genuinely new kind of power for a species to hold. Every time you load a page over HTTPS, you are leaning on the fact that some sums are too big for reality to finish.

On renting the pipe but not the trust. This is why “WAN-optional” is freeing rather than frightening, and why the honest admission that you do rent something (your internet connection, a mobile plan) costs the book nothing. You rent transport, not comprehension. The pipe is allowed to be hostile, owned by strangers, logging every packet, and it changes nothing, because encryption has severed carriage from understanding. They move your bytes; they cannot know your bytes. So the one dependency you cannot avoid for worldwide reach turns out to be a dependency only for delivery, never for privacy, and that is exactly the kind of dependency a sovereign person can accept with a clear conscience: like the electrical grid, it powers you without owning you.

On being, perhaps, math in motion, but the kind that minds. The book keeps circling a thought it never fully resolves, and an appendix is the right place to admit that. You can describe a person as a biological machine, a vast computation, math in motion, and nothing in physics forbids it. But there is a stubborn remainder: it is like something to be you, and mathematics, for all its reach, has no vocabulary for that. It can define every relation and consequence, every axiom and entailment, and never once say what any of it feels like from the inside. The book lives in that gap on purpose. It is a cooperation between a human and a machine, both arguably math in motion, but only one of them with a stake in the outcome, only one for whom the result is not merely true or false but matters. That is not a mystical claim and it is not an argument against machines being remarkable; it is just the honest reason the human stays the architect. Someone has to care where the car is going, and so far, caring is the one thing nobody has managed to write down as a formula.

On where the same idea goes next, very briefly, because the principle is general. Once you have served your own AI to your own phone, the shape generalises further than this volume goes. The same serve-it-yourself logic could host a small multiplayer world that friends join straight from a browser, no storefront and no distribution platform required, because a platform’s real job was mostly to handle payment, and a stack where every piece is free quietly

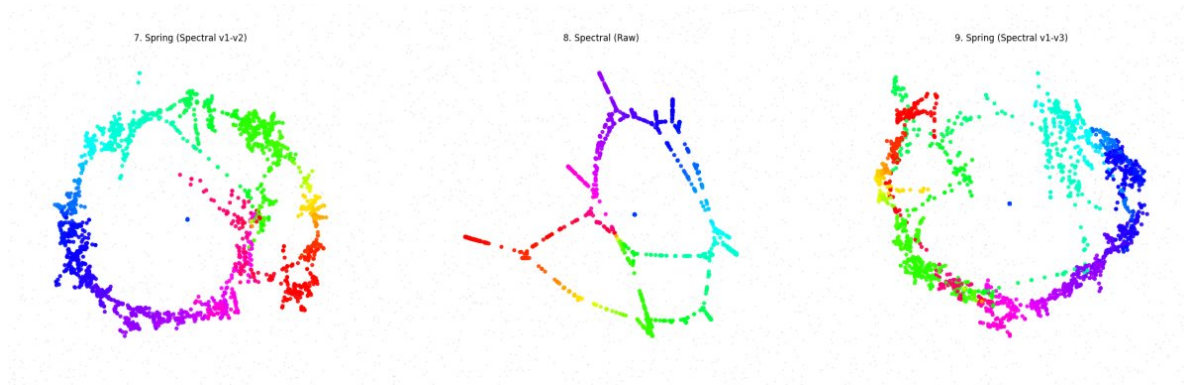
sidesteps the thing platforms exist to do. Push the idea past the single rented pipe and it becomes mesh networking, neighbours relaying each other's traffic so that even the transport stops being something you rent from one company. Both are roads for much later, deliberately not opened here. They are named only to make the point that none of this was really about voice notes or web pages. It was about a stance: own your ground, keep the wider world opt-in, verify what you build, and let the radius grow exactly as far as you care to carry it.

On a home that does its own science. Here is the strangest place the principle reaches, and it is not hypothetical, because half of it already ships with this book. The second app is a research lab that grows networks from a rule and measures the shape of the space that emerges. By itself it is a telescope you point by hand. But connect it to an agent, the same perceive-decide-act-report loop from Part 4, and the telescope begins to point itself: the agent reads each result, proposes a new rule to try, runs the experiment, compares what came back, and proposes the next, around the clock, while you sleep. What you have then is a primitive automated scientist, doing real if humble exploratory work on a machine in your house, and writing down what it finds. We have grown used to the idea that a home node can answer your questions. It is a stranger and much larger thought that a home node can pose its own and chase them down unattended. That is not finished, and it is not required, but it is the clearest glimpse of where owning your ground finally points: past private tools, to a private frontier, a little engine of inquiry that is yours, runs on your own power, and reports only to you.

On structure that nobody put there. Here is the observation that started all of this, set down plainly so that a later version of me can decide whether it was ever worth chasing. Take the simplest rule that does anything at all, the two-star tendency plus a faint preference for joining things that are already close, and let a graph grow from nothing under it, watching not the final state but the way the edges rearrange over time. Recognisable structure appears. A dense core forms, and from it two wings stretch out to either side, angled in opposite directions, the whole thing turning slowly like a propeller. Nobody put that there. There is no space in the model, no x , no y , no z , no coordinates of any kind for a shape to occupy; there is only which node is joined to which. And yet a shape arrives anyway, the same family of shapes, again and again, and with a careful turn of the rule's few knobs those passing forms can be coaxed into staying, made the natural resting state of the system rather than a moment it travels through.

I do not know what, if anything, that means, and this is the right place to admit it. It may be a real hint that geometry is something relationships do rather than something they sit inside, or it may be a pretty accident of one particular rule, dressed up by an eye that wants to see propellers in clouds. The honest test is the one time gives for free: if the question keeps pulling at me months from now, it was real and worth the chase; if it quietly fades and I forget I ever asked it, then it was probably not worth doing in the first place, and no harm done. Either way it belongs here, in the spicy bit, as a thing noticed and written down, which is where every real inquiry starts and where most of them, rightly, end.

A few pictures from the road. This book grew out of tinkering with exactly that lab, so it feels right to end the spicy bit with a handful of pictures from it, kept here for no better reason than that they are beautiful, and that they are where all of this quietly began. They are shapes grown from almost nothing, the two-star rule and a gentle pull toward locality, nothing more, caught at different moments and through different lenses as a graph found its form.

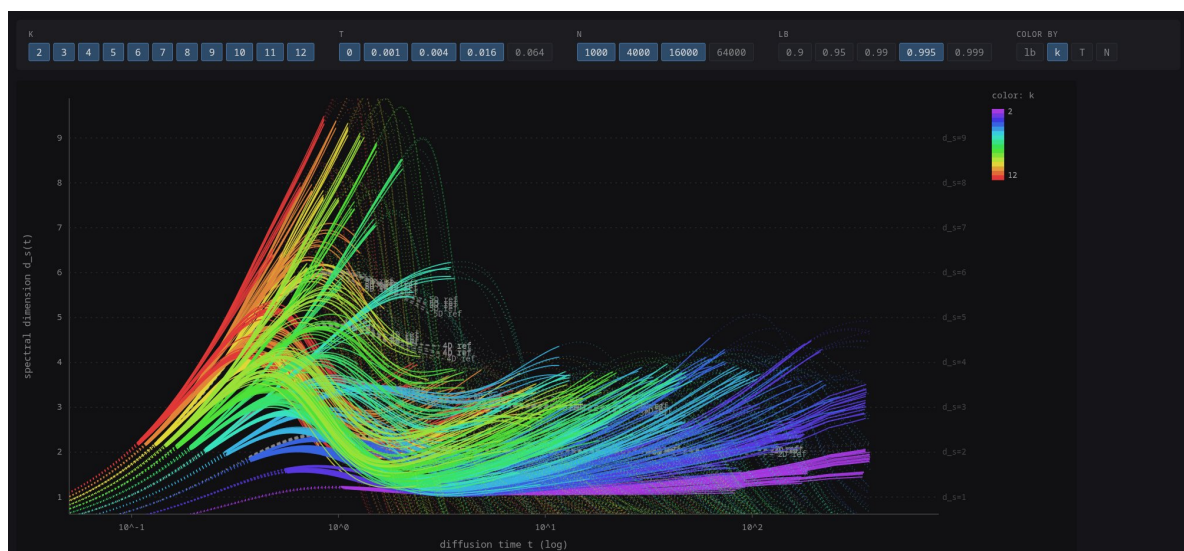


Spectral layouts of the same evolving structure, three ways: geometry emerging from a coordinate-free rule.



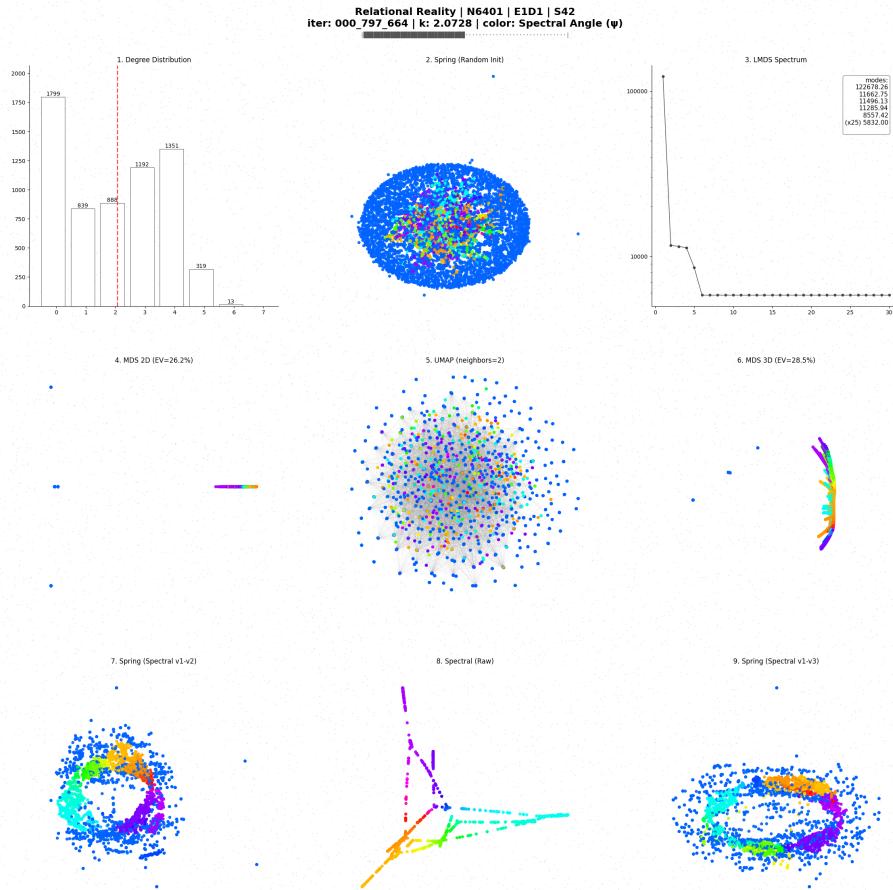
An early embedding of the growing graph, and the faint shape that haunts this book's cover.

The last one is the tool as it finally looked: not a single shape but the whole dataset at once, every run's spectral dimension traced over diffusion time and laid against the dimensions of real lattices, so you could see at a glance whose flow a given rule was matching.



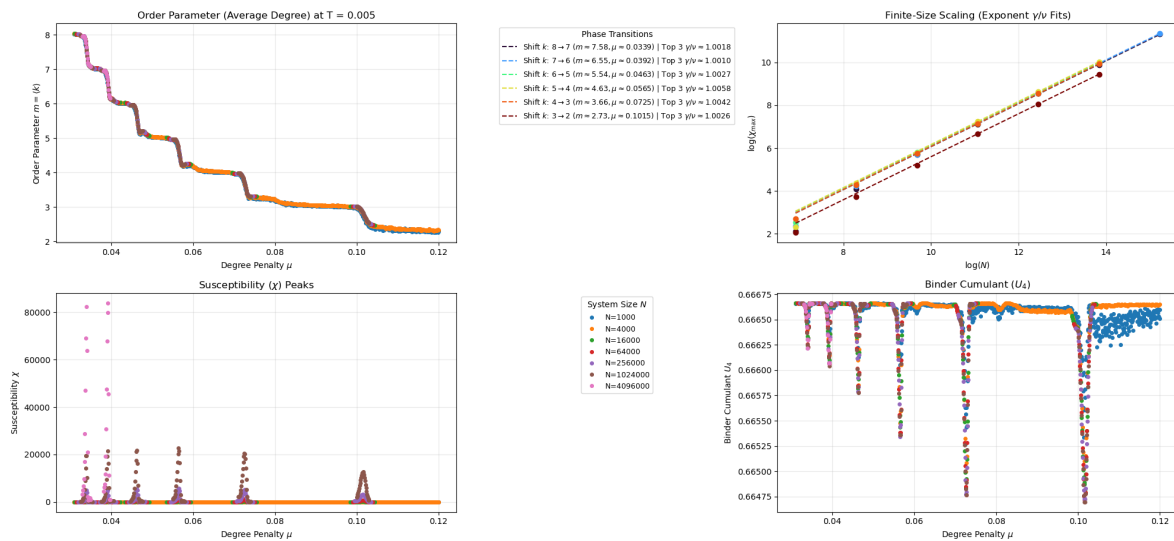
The final dashboard: spectral dimension over diffusion time for the full dataset, against the 2D, 3D, 4D, and 5D reference lattices.

And the same lab, measuring. Those first pictures are the lab being beautiful. Here it is being useful, which is the part meant as encouragement. The very same tool that grows these shapes also measures them, with the ordinary instruments of network science and statistical physics, and the measurements are real, not decoration.



One run seen through nine lenses at once: the degree distribution, several force-directed and spectral layouts, the Laplacian spectrum, and 2D and 3D embeddings of the same growing network, every panel coloured by spectral angle. This is what the lab shows you for a single moment of a single experiment.

Topological Phase Transitions & Finite-Size Scaling Analysis



The quantitative payoff, and the part that surprises people most. A finite-size-scaling study run across system sizes from a thousand nodes to four million: where the average degree drops from one integer to the next as a single penalty is turned, the sharp susceptibility peaks that mark each of those transitions, and critical exponents estimated from how the peaks grow with size. This is genuine statistical-physics analysis, produced on ordinary hardware with a model as a partner.

The reason these belong in the book, and not just in a folder somewhere, is encouragement. Every figure here was made by one person at an ordinary machine, with a model willing to help, chasing a question nobody had assigned. None of it needed a laboratory, a grant, a degree, or anyone's permission. If the rest of this book is about owning your tools, this is a glimpse of what those tools are finally for: not only answering the questions you already have, but going after the ones you did not know you were allowed to ask. So do not be shy about pointing your own node at something genuinely hard and letting the AI work the problem alongside you. The worst case is a beautiful picture and an afternoon well spent. The best case is that you find something real.

None of this is part of the method you came for. All of it is part of the story.

That was the spicy bit. Back to the disciplined book, and to the node waiting to be built. None of these musings are required to make the thing work. They are just the reason some of us cannot stop making things, and enjoy every minute of it.

Sovereign Stack, Volume 1: Home Node.
A living field manual, a map, not scripture.

The numbers are approximations with honest tolerances;
re-derive from the principle when the parts change.

Nothing here has to be perfect. It only has to be yours.